

实验报告

姓名	张泽渊	学号	1120200325	手机	15903129725
学院	计算机学院	班级	08012101	指导教师	
项目名称	“OpenHarmony 应用开发”专题：QQ 聊天软件模仿应用开发				

本篇实验报告的篇幅较长，推荐结合目录进行查看。

目录

一、前言	1
二、项目实验环境	2
三、实验过程	2
3.1 环境搭建	2
3.2 软件架构	3
3.3 编码实现	4
3.3.1 首页页面实现	4
3.3.2 注册页面实现	6
3.3.3 登录页面实现	9
3.3.4 主页面实现	13
3.3.5 导航栏页面实现	17
3.3.6 主页面内消息 TAB 组件实现	19
3.3.7 optionItem 页面实现	24
3.3.8 主页面内联系人 TAB 组件实现	26
3.3.9 主页面内动态 TAB 组件实现	33
3.3.10 个人页面实现	38
3.3.11 设置页面实现	42
3.3.12 聊天页面实现	44
3.3.13 好友资料页面实现	53
3.3.14 空间动态页面实现	56
3.3.15 备注	58
四、实验结果	59
五、实验感想	61

一、前言

本次实验是对“OpenHarmony 应用开发”专题的深入实践，实验内容为使用 DevEco Studio 软件，应用 ArkTS 语言，开发一个基于 OpenHarmony 的模仿 QQ 界面的聊天软件。

通过以下链接可以快速了解实验成果：

• 实验成果代码：<https://github.com/vhthree/QQzzy>

备注：代码仓库将在 2024 年 9 月 30 日 24:00 之后开放。

• 软件演示视频：<https://www.bilibili.com/video/BV1eJWKeoEqm/>

二、项目实验环境

操作系统：Windows10 操作系统

IDE：DevEco Studio

语言环境：ArkTS

编译器：方舟编译体系

Compile SDK：12

Compatible SDK：12

API 版本：12

三、实验过程

3.1 环境搭建

- 安装 IDE—DevEco Studio

下载链接：<https://developer.huawei.com/consumer/cn/download/>

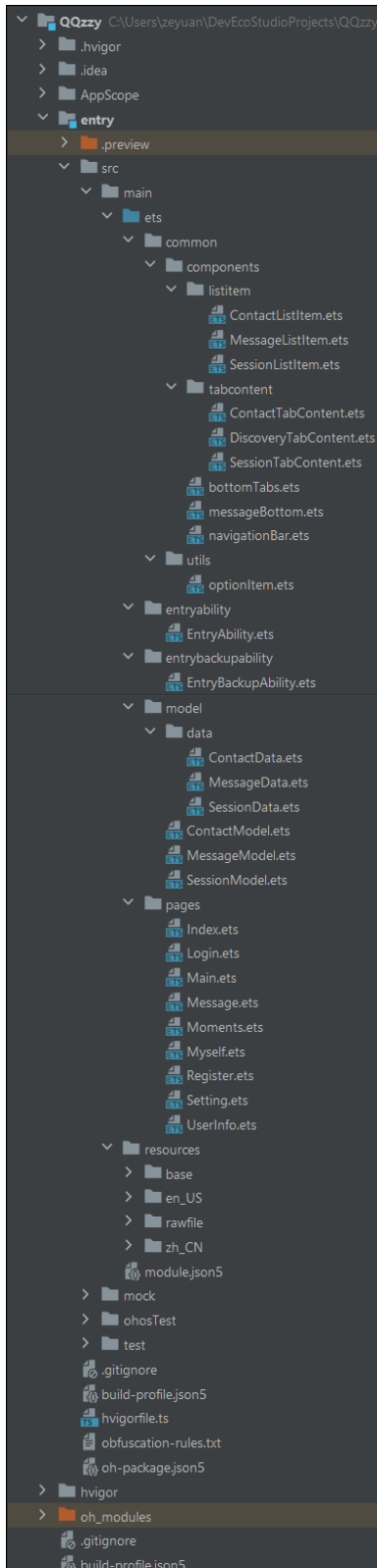
- sdk 安装

下载链接：

<http://ci.openharmony.cn/workbench/cicd/dailybuild/detail/component>

• 在构建项目前，需要先将系统类型从 HarmonyOS 切换到 OpenHarmony，需要将 build-file.json5 文件"products"部分的代码改为下图所示。

```
"products": [  
  {  
    "name": "default",  
    "signingConfig": "default",  
    "compileSdkVersion": 12,  
    "compatibleSdkVersion": 12,  
    "targetSdkVersion": 12,  
    "runtimeOS": "OpenHarmony"  
  }  
],
```



3.2 软件架构

在 OpenHarmony 的开发框架下，开发者需要编写的文件位于 `src` 目录的 `main` 目录下，本次实验中 `main` 目录的架构如下。

`src > main > ets`：用于存放 ArkTS 源码。

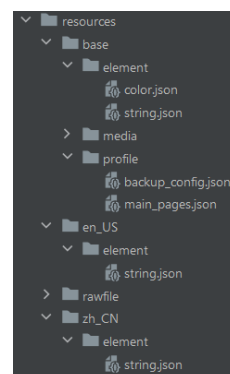
`src > main > ets > common`：包含可以重复使用的各个构件，比如导航栏、用户资料卡片、消息卡片、设置项卡片等等。

`src > main > ets > entryability`：设置应用/服务的入口，本次实验设置为 `pages/Index` 页面。

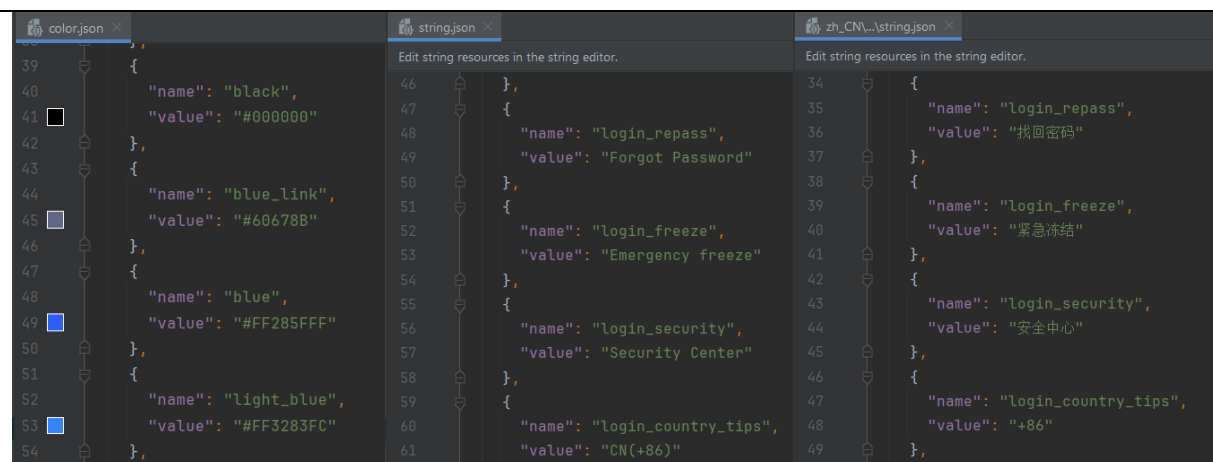
`src > main > ets > model`：存放软件的用户数据，比如好友信息、聊天记录信息等等，其中的 `data` 目录包含定义数据时使用的类文件，以及调用用户数据的接口。

`src > main > ets > pages`：存放供用户使用的应用页面。页面在 `pages` 目录下配置后，需要打开“`entry > src > main > resources > base > profile`”，在 `main_pages.json` 文件中的“`src`”下配置页面的路由信息，方便实现各个页面之间的跳转。

`src > main > resources`：用于存放应用/服务所用到的资源文件，如图形、多媒体、字符串、布局文件等，其目录结构如下。



`element` 目录下的 `color.json` 文件储存了应用使用的颜色信息，可供所有页面共同调用同一个颜色，方便统一不同页面的颜色风格；`string.json` 文件包含软件内的文字信息，便于软件切换语言，本软件的语言可在中文版本和英文版本之间切换。



media 目录存放组装各种页面的图片素材，如各种图标。

rawfile 目录存放用户的图片数据，比如用户头像。

src>main>module.json5: 模块配置文件。主要包含 HAP 包的配置信息、应用/服务在具体设备上的配置信息以及应用/服务的全局配置信息。

这些文件包含了本次实验需要编写的全部内容。

3.3 编码实现

3.3.1 首页页面实现

3.3.1.1 涉及的文件

src > main > ets > pages > Index.ets

3.3.1.2 文件源代码

```

1. // 该组件为入口页面
2. import router from '@ohos.router';
3. @Entry
4. @Component
5. struct Index {
6.   build() {
7.     Stack({alignContent: Alignment.Bottom}){
8.       Image($r('app.media.welcome'))
9.       .height('50%')
10.      .margin({ top: '0.00vp', right: '0.00vp', bottom: '60.00%', left: '0.00vp'
11.      })
12.      Flex({justifyContent: FlexAlign.SpaceBetween}){
13.        Button($r('app.string.splash_register') , { type: ButtonType.Normal })
14.        .width(150)
15.        .height(50)
16.        .backgroundColor($r("app.color.white"))
17.        .fontSize(20)
18.        .fontColor($r("app.color.black"))
19.        .borderRadius(10)
20.        .onClick(() => {
21.          router.pushUrl({ url: 'pages/Register' })

```

```

21.         })
22.         .padding({ top: '0.00vp', right: '16.00vp', bottom: '0.00vp', left: '16.0
    0vp' })
23.         .borderWidth('2.00vp')
24.         .borderColor('#7a000000')
25.         Button($r('app.string.splash_login'), { type: ButtonType.Normal })
26.         .width(150)
27.         .height(50)
28.
29.         .fontSize(20)
30.         .fontColor($r("app.color.white"))
31.         .borderRadius(10)
32.         .onClick(() => {
33.             router.pushUrl({ url: 'pages/Login' })
34.         })
35.         .backgroundImageSize({ width: '', height: '' })
36.         .backgroundColor('#ff009aff')
37.     }
38.     .width('100%')
39.     .height(90)
40.     .padding(20)
41. }
42. .width('100%')
43. .height('100%')
44. }
45. }

```

3.3.1.3 布局结构

使用 Stack 容器实现页面布局，alignContent 属性设置为 Alignment.Bottom，将内容对齐到底部。

页面中心展示一张 QQ 的欢迎图片，使用 Image() 组件，图片资源从应用资源文件中获取，设置图片高度为页面的 50%。

页面整体使用百分比布局和相对单位(vp)，确保应用在不同屏幕尺寸下具有良好的响应性。

按钮的边距和填充通过 padding 和 margin 属性控制。

3.3.1.4 按钮设计

页面底部通过 Flex 容器创建了两个按钮，分别为“新用户”和“登录”按钮。FlexAlign.SpaceBetween 用于在两个按钮之间均匀分布空间。

- 注册按钮：

使用 Button() 组件创建，显示“新用户”字样。

通过 router.pushUrl() 函数绑定点击事件，跳转到注册页面。

- 登录按钮：

同样使用 Button() 组件创建，显示“登录”字样。

点击按钮会跳转到登录页面。

3.3.1.5 关键函数

\$r('app.media.welcome') 表示从 media 资源文件内寻找名为 welcome 的图片。



`$r('app.string.splash_login')`表示在 `string.json` 文件中寻找 `splash_login` 对应的文字，方便快捷切换软件的语言。

通过 `router.pushUrl()` 方法进行页面路由切换。

3.3.1.6 页面功能

页面实现了应用启动后首页的界面，包括显示欢迎图片、提供注册和登录按钮，并通过点击按钮实现页面跳转。

3.3.2 注册页面实现

3.3.2.1 涉及的文件

`src > main > ets > pages > Register.ets`

3.3.2.2 文件源代码

```
1. // 注册页面
2. import router from '@ohos.router';
3. @Extend(Text) function text_style() {
4.     .width('25%')
5.     .fontFamily($r('sys.float.ohos_id_text_size_body2'))
6.     .fontSize(15)
7.     .maxLines(1)
8. }
9. @Extend(TextInput) function input_style() {
10.    .width('75%')
11.    .fontFamily($r('sys.float.ohos_id_text_size_body2'))
12.    .fontSize(15)
13.    .placeholderFont({size: 15 , family: $r('sys.float.ohos_id_text_size_body2')})
14.    .placeholderColor($r('app.color.grey3'))
15.    .caretColor($r('app.color.green0'))
16.    .backgroundColor(Color.White)
17. }
18. @Extend(Divider) function divider_style() {
19.    .width('90%')
20.    .color($r('app.color.grey2'))
21.    .strokeWidth(0.8)
22. }
23. @Entry
24. @Component
25. struct Register {
26.    build() {
27.        Stack({}) {
28.            Image($r('app.media.register'))
29.            .height('100%')
30.            Flex({ direction: FlexDirection.Column, alignItems: ItemAlign.Start, justifyContent: FlexAlign.SpaceBetween }) {
31.                Image($r('app.media.ic_back'))
```

```

32.         .margin({ top: '10.00vp', right: '0.00vp', bottom: '0.00vp', left: '10.00
vp' })
33.         .width(26)
34.         .height(26)
35.         .onClick(() => {
36.             router.back()
37.         })
38.         Text('').height(100)
39.         Text('欢迎注册 QQ')
40.         .fontSize(36)
41.         .fontFamily($r('sys.float.ohos_id_text_size_headline1'))
42.         .margin({
43.             top: '20.00vp',
44.             right: '0.00vp',
45.             bottom: '0.00vp',
46.             left: '30.00vp'
47.         })
48.         .fontColor($r('app.color.white'))
49.         Text('每一天，乐在沟通')
50.         .fontSize(23)
51.         .fontFamily($r('sys.float.ohos_id_text_size_headline7'))
52.         .margin({
53.             top: '20.00vp',
54.             right: '0.00vp',
55.             bottom: '0.00vp',
56.             left: '30.00vp'
57.         })
58.         .fontColor($r('app.color.white'))
59.         Flex({ direction: FlexDirection.Column, alignItems: ItemAlign.Center }) {
60.             Row() {
61.                 TextInput({ placeholder: $r('app.string.register_nickname') })
62.                 .input_style()
63.                 .borderRadius('10.00vp')
64.                 .width('98%')
65.             }.width('90%').height(60).margin({ bottom: 4, top: 4 })
66.             .borderWidth('0.00vp')
67.             Row() {
68.                 TextInput({ placeholder: $r('app.string.register_password') })
69.                 .input_style()
70.                 .type(InputType.Password)
71.                 .borderRadius('10.00vp')
72.                 .width('98%')
73.             }.width('90%').height(60).margin({ bottom: 4 })
74.             Row() {
75.                 TextInput({ placeholder: $r('app.string.login_country_tips') })
76.                 .input_style()

```

```

77.         .width('20%')
78.         .borderRadius('10.00vp')
79.         .margin({ right: 10 })
80.         TextInput({ placeholder: $r('app.string.login_phone_tips') })
81.         .input_style()
82.         .type(InputType.Number)
83.         .borderRadius('10.00vp')
84.     }.width('90%').height(60).margin({ bottom: 4 })
85.
86. }.width('100%').height(200)
87. Flex({ direction: FlexDirection.Column, justifyContent: FlexAlign.End, align
nItems: ItemAlign.Center }) {
88.     Row(){
89.         Checkbox({ name: 'checkbox2' })
90.         .select(false)
91.         .selectedColor(0x39a2db)
92.         .shape(CheckBoxShape.ROUNDED_SQUARE)
93.         .borderWidth('0.50vp')
94.         .backgroundColor(Color.White)
95.         .borderRadius('5.00vp')
96.         Text() {
97.             Span($r('app.string.register_privacy1'))
98.             .fontColor($r('app.color.white'))
99.             .fontSize(15)
100.            Span($r('app.string.register_privacy2'))
101.            .fontSize(15)
102.            .fontColor($r('app.color.blue'))
103.            .fontFamily($r('sys.float.ohos_id_text_size_body2'))
104.            Span($r('app.string.register_privacy3'))
105.            .fontColor($r('app.color.white'))
106.            .fontSize(15)
107.            Span($r('app.string.register_privacy4'))
108.            .fontSize(15)
109.            .fontColor($r('app.color.blue'))
110.            .fontFamily($r('sys.float.ohos_id_text_size_body2'))
111.        }
112.    }
113.    Button($r('app.string.login_button'), { type: ButtonType.Normal })
114.        .width(170)
115.        .height(50)
116.        .backgroundColor($r("app.color.light_blue"))
117.        .fontSize(18)
118.        .fontColor($r("app.color.white"))
119.        .borderRadius(8)
120.        .margin({ top: 30, bottom: 100 })
121.        .onClick(() => {

```



```

122.         router.pushUrl({ url: 'pages/Main' })
123.     })
124.     }.width('100%').layoutWeight(1)
125.     }.width('100%').height('100%')
126. }
127. }
128. }

```

3.3.2.3 布局结构

注册页面采用了 Stack 和 Flex 容器实现页面布局：

Stack 容器负责页面的整体布局：背景图片覆盖了整个页面，增加页面的视觉效果。

Flex 容器使用 FlexDirection.Column 设置纵向排列：顶部展示返回按钮，按钮样式为向左的箭头；中央部分放置昵称、密码、手机号码输入框；底部放置注册按钮和服务协议说明。通过 alignItems 和 justifyContent 属性进行对齐，保证页面布局的整齐与合理性。

3.3.2.4 按钮设计

- 返回按钮：

使用 Image() 组件渲染返回图标，图标为向左的箭头，通过 onClick() 函数为其绑定点击事件，使用户点击后可以返回到上一个页面（首页页面）。

- 注册按钮：

使用 Button() 组件创建。点击按钮后会跳转到主页面，模拟用户注册流程的核心功能。

3.3.2.5 关键函数

text_style()、input_style()、divider_style() 等函数：

通过 @Extend() 修饰符定义，分别用于扩展文本、输入框、分隔符的样式，简化了代码的重复编写，同时保持页面元素样式的一致性。

3.3.2.6 页面功能

页面顶部展示背景图片，并使用大号字体展示欢迎注册的文字，引导用户进行注册操作。

多个 TextInput() 组件用于输入昵称、密码、手机号码等信息，输入框的样式通过 input_style() 函数进行设置。使用 Checkbox() 组件添加服务协议的可复选框，用户选择同意协议后才能点击注册按钮进行下一步操作。

注册按钮的点击事件会触发页面跳转，带用户进入主界面。



3.3.3 登录页面实现

3.3.3.1 涉及的文件

src > main > ets > pages > Login.ets

3.3.3.2 文件源代码

```

1. // 该组件为登录页面
2. import router from '@ohos.router';
3. @Extend(Text)
4. function text_style() {
5.     .width('25%')
6.     .fontFamily($r('sys.float.ohos_id_text_size_body2'))

```

```

7.     .fontSize(15)
8.     .maxLines(1)
9. }
10. @Extend(Text)
11. function link_text_style(fontSize: number) {
12.     .fontSize(fontSize)
13.     .fontColor($r('app.color.blue_link'))
14.     .fontFamily($r('sys.float.ohos_id_text_size_body2'))
15.     .maxLines(1)
16. }
17. @Extend(TextInput)
18. function input_style() {
19.     .width('75%')
20.     .fontFamily($r('sys.float.ohos_id_text_size_body2'))
21.     .fontSize(15)
22.     .placeholderFont({ size: 15, family: $r('sys.float.ohos_id_text_size_body2') })
23.     .placeholderColor($r('app.color.grey3'))
24.     .caretColor($r('app.color.green0'))
25.     .backgroundColor(Color.White)
26. }
27. @Extend(Divider)
28. function divider_style() {
29.     .width('90%')
30.     .color($r('app.color.grey2'))
31.     .strokeWidth(0.8)
32. }
33. @Entry
34. @Component
35. struct Login {
36.     build() {
37.         Stack({}) {
38.             Image($r('app.media.gradualchange'))
39.             .height('100%')
40.             Flex({ direction: FlexDirection.Column, alignItems: ItemAlign.Start, justifyC
ontent: FlexAlign.SpaceBetween }) {
41.                 Image($r('app.media.ic_back'))
42.                 .margin({ top: '10.00vp', right: '0.00vp', bottom: '0.00vp', left: '10.00
vp' })
43.                 .width(26)
44.                 .height(26)
45.                 .onClick(() => {
46.                     router.back()
47.                 })
48.                 Flex({ direction: FlexDirection.Column, alignItems: ItemAlign.Center }) {
49.                     Image($r('app.media.qq'))
50.                     .height('50%')

```

```

51.         .width('auto')
52.     Row() {
53.         TextInput({ placeholder: 'QID/邮箱' })
54.         .input_style()
55.         .borderRadius('5.00vp')
56.         .width('90%')
57.     }.height(60).margin({ bottom: 4, top: 4 })
58.     Row() {
59.         TextInput({ placeholder: '输入 QQ 密码' })
60.         .input_style()
61.         .type(InputType.Password)
62.         .borderRadius('5.00vp')
63.         .width('90%')
64.     }.height(60).margin({ bottom: 4 })
65.     Row() {
66.         Checkbox({ name: 'checkbox1' })
67.         .select(false)
68.         .selectedColor(0x39a2db)
69.         .shape(CheckBoxShape.ROUNDED_SQUARE)
70.         .backgroundColor(Color.White)
71.         .borderRadius('5.00vp')
72.         Text() {
73.             Span($r('app.string.register_privacy1'))
74.             .fontColor($r('app.color.white'))
75.             .fontSize(15)
76.             Span($r('app.string.register_privacy2'))
77.             .fontSize(15)
78.             .fontColor($r('app.color.blue'))
79.             .fontFamily($r('sys.float.ohos_id_text_size_body2'))
80.             Span($r('app.string.register_privacy3'))
81.             .fontColor($r('app.color.white'))
82.             .fontSize(15)
83.             Span($r('app.string.register_privacy4'))
84.             .fontSize(15)
85.             .fontColor($r('app.color.blue'))
86.             .fontFamily($r('sys.float.ohos_id_text_size_body2'))
87.         }
88.     }
89.     Button($r('app.string.splash_login'), { type: ButtonType.Normal })
90.         .width('90%')
91.         .height(50)
92.         .backgroundColor($r("app.color.light_blue"))
93.         .fontSize(18)
94.         .fontColor($r("app.color.white"))
95.         .borderRadius(8)
96.         .margin({ top: 30 })

```

```

97.         .onClick(() => {
98.             router.pushUrl({ url: 'pages/Main' })
99.         })
100.    }
101.    .width('100%')
102.    .height(400)
103.    .align(Alignment.Center)
104.    .margin({
105.        top: '50.00vp',
106.        right: '0.00vp',
107.        bottom: '10.00vp',
108.        left: '0.00vp'
109.    })
110.    Flex({ direction: FlexDirection.Column, justifyContent: FlexAlign.End, alignItems: ItemAlign.Center }) {
111.        Flex({ direction: FlexDirection.Row, alignItems: ItemAlign.End, justifyContent: FlexAlign.Center }) {
112.            Text($r('app.string.login_repass'))
113.            .link_text_style(12)
114.            .margin({ right: 5 })
115.            Text('|')
116.            .link_text_style(12)
117.            .margin({ right: 5 })
118.            Text($r('app.string.login_freeze'))
119.            .link_text_style(12)
120.            .margin({ right: 5 })
121.            Text('|')
122.            .link_text_style(12)
123.            .margin({ right: 5 })
124.            Text($r('app.string.login_security'))
125.            .link_text_style(12)
126.        }.width('100%').padding({ bottom: 30, top: 60 })
127.    }.width('100%').layoutWeight(1)
128.    }
129.    }
130.    }
131.    }

```

3.3.3.3 布局结构

登录页面使用了 Stack 和 Flex 容器进行布局：

Stack 容器负责页面的整体布局，背景图片覆盖整个页面。

Flex 容器内部采用了 FlexDirection.Column 纵向排列，分别容纳了返回按钮、应用 Logo、输入框、登录按钮等元素。通过 alignItems 和 justifyContent 属性对内容进行对齐与空间分布设置。

3.3.3.4 按钮设计

- 返回按钮：

Image() 组件渲染一个返回图标，图标为向左的箭头，通过 onClick() 函数绑定点击事件，使用户点击后可以返回上一个页面（首页页面）。

- 登录按钮：

使用 Button() 组件创建，显示“登录”字样。

点击按钮后，页面跳转到主界面（pages/Main），实现了核心的登录功能。

3.3.3.5 关键函数

text_style()、link_text_style()、input_style() 等函数：

通过 @Extend() 修饰符定义，负责对页面中的文本、输入框、分隔符等组件进行样式扩展，便于代码复用和界面风格的一致性。

3.3.3.6 页面功能

背景图片铺满整个页面，页面顶部展示应用 Logo，使界面更具视觉吸引力。

两个 TextInput() 组件分别用于输入用户的账号（如 QID/邮箱）和密码，输入框的样式通过 input_style() 进行设置。密码输入框添加了 type(InputType.Password) 属性，确保输入内容为不可见的密码格式。

登录按钮的点击事件触发后，页面将跳转到主页面，模拟典型的登录流程。



3.3.4 主页面实现

3.3.4.1 涉及的文件

src > main > ets > pages > Main.ets

src > main > ets > common > components > bottomTabs.ets

src > main > ets > common > components > tabcontent > SessionTabContent.ets

src > main > ets > common > components > tabcontent > ContactTabContent.ets

src > main > ets > common > components > tabcontent > DiscoveryTabContent.ets

3.3.4.2 文件源代码

Main.ets 代码：

```
1. // 主界面
2. import { BottomTabs } from '../common/components/bottomTabs'
3. import { SessionTabContent } from '../common/components/tabcontent/SessionTabContent'
4. import { ContactTabContent } from '../common/components/tabcontent/ContactTabContent'
5. import { DiscoveryTabContent } from '../common/components/tabcontent/DiscoveryTabContent'
6. @Entry
7. @Component
8. struct Main {
```

```

9.     private controller: TabsController = new TabsController()
10.    @State title: Resource[] =
11.        [${r("app.string.title_qq")}, ${r('app.string.title_contact')}, ${r("app.string.titl
            e_moment")}]
12.    @State bottomTabIndex: number = 0
13.    build() {
14.        Flex({ direction: FlexDirection.Column, alignItems: ItemAlign.End, justifyConte
            nt: FlexAlign.End }) {
15.            Tabs({
16.                barPosition: BarPosition.End,
17.                index: this.bottomTabIndex,
18.                controller: this.controller
19.            }) {
20.                TabContent() {
21.                    SessionTabContent()
22.                }
23.                TabContent() {
24.                    ContactTabContent()
25.                }
26.                TabContent() {
27.                    DiscoveryTabContent()
28.                }
29.            }
30.            .onChange((index: number) => {
31.                this.bottomTabIndex = index
32.            })
33.            .vertical(false)
34.            .barHeight(0)
35.            .width('100%')
36.            .scrollable(true)
37.            BottomTabs({ controller: this.controller, bottomTabIndex: this.bottomTabIndex
                })
38.        }
39.        .backgroundColor(${r('app.color.white')})
40.        .width('100%').height('100%')
41.    }
42. }

```

bottomTabs.ets 代码:

```

1. //该组件为底部标签栏，用于主界面底部。
2. class TabItem{
3.     imgSrcNormal: Resource
4.     imgSrcPress: Resource
5.     tabText: Resource

```

```

6.     constructor(imgSrcNomral: Resource, imgSrcPress: Resource , tabText: Resource) {
7.         this.imgSrcNomral = imgSrcNomral
8.         this.imgSrcPress = imgSrcPress
9.         this.tabText = tabText
10.    }
11. }
12. function getTabItems(): Array<TabItem> {
13.     let itemsArray: Array<TabItem> = []
14.     itemsArray.push(new TabItem($r('app.media.message_normal') , $r('app.media.message_press') , $r('app.string.bottom_tab_message') ))
15.     itemsArray.push(new TabItem($r('app.media.contacts_normal') , $r('app.media.contacts_press') , $r('app.string.bottom_tab_contact') ))
16.     itemsArray.push(new TabItem($r('app.media.discovery_normal') , $r('app.media.discovery_press') , $r('app.string.bottom_tab_discovery') ))
17.     return itemsArray;
18. }
19. @Component
20. export struct BottomTabs {
21.     private tabSrc: number[] = [0, 1, 2]
22.     private controller: TabsController = new TabsController()
23.     @Link bottomTabIndex: number
24.     private tabItemArray: Array<TabItem> = getTabItems()
25.     build() {
26.         Flex({ direction: FlexDirection.Row, alignItems: ItemAlign.Center, justifyContent: FlexAlign.SpaceEvenly }) {
27.             ForEach(this.tabSrc, (item: number) => {
28.                 Column() {
29.                     Image((this.bottomTabIndex == item) ? this.tabItemArray[item].imgSrcPress : this.tabItemArray[item].imgSrcNomral)
30.                         .objectFit(ImageFit.Contain)
31.                         .width(30).height(30)
32.                     Text(this.tabItemArray[item].tabText)
33.                         .fontSize(14)
34.                         .fontFamily($r('sys.float.ohos_id_text_size_body2'))
35.                         .margin({ top: 3 })
36.                 }
37.                 .onClick(() => {
38.                     this.controller.changeIndex(item)
39.                 })
40.             }, (item: number) => item.toString())
41.         }
42.         .width('100%').height(74)
43.         .backgroundColor($r('app.color.white'))
44.     }

```

3.3.4.3 布局结构

• 主界面 (Main.ets):

使用 Flex 容器实现页面布局，设置了纵向排列 (FlexDirection.Column) 以便内容从上到下布局。

页面通过 Tabs 组件设置多个标签页，用户可以在不同的页面内容之间切换。每个标签页对应不同的内容组件，依次是消息 (SessionTabContent.ets)、联系人 (ContactTabContent.ets)、动态 (DiscoveryTabContent.ets)。

标签栏的位置设置在页面底部，使用 BottomTabs 组件来显示底部导航栏。

• 底部标签栏 (bottomTabs.ets):

标签栏使用 Flex 容器进行横向布局 (FlexDirection.Row)，并通过 FlexAlign.SpaceEvenly 来均匀分布每个标签项。

每个标签项包括一个图标和文本，分别通过 Image 和 Text 组件进行渲染。标签栏的宽度设置为页面的 100%，高度为 74vp，保证整个标签栏占据页面底部固定的空间。



3.3.4.4 按钮设计

• 主界面 (Main.ets):

标签页通过 Tabs 组件实现，其中每个标签页都有相应的 TabContent() 方法。点击标签页会调用 onChange() 事件，更新当前激活的标签索引 (bottomTabIndex)。

• 底部标签栏 (bottomTabs.ets):

底部标签项通过 ForEach() 循环生成，包含一个图标和文字。点击标签项时，会调用 controller.changeIndex() 方法，实现标签页的切换。

图标和文字样式会根据当前选择的标签项进行动态变化，确保用户可以直观地看到激活的标签。

3.3.4.5 关键函数

• 主界面 (Main.ets):

Tabs(): 用于渲染主界面中的多个标签页，允许用户在不同的功能模块之间切换。

onChange(): 事件处理函数，用于捕捉标签切换动作，并更新底部标签栏的状态。

• 底部标签栏 (bottomTabs.ets):

getTabItems(): 用于初始化底部标签栏的内容，包括每个标签项的图片资源和文字内容。

controller.changeIndex(): 用于更新当前激活的标签索引，以实现标签页的切换。

3.3.4.6 页面功能

• 主界面 (Main.ets):

主界面实现了一个基础的聊天应用框架，包含消息 (SessionTabContent.ets)、联系人 (ContactTabContent.ets)、动态 (DiscoveryTabContent.ets) 三个主要功能模块。用户可以通过点击底部标签栏在这些模块之间切换。

该界面通过 Tabs 和 BottomTabs 组件实现了标签页的切换和底部导航功能。

• 底部标签栏 (bottomTabs.ets):

底部标签栏是应用主界面的导航工具，用户可以通过点击不同的标签项，快速访问消息 (SessionTabContent.ets)、联系人 (ContactTabContent.ets)、动态 (DiscoveryTabContent.ets) 三个页面。每个标签项被选择时会改变颜色以便用户查看。

3.3.5 导航栏页面实现

3.3.5.1 涉及的文件

src > main > ets > common > components > navigationBar.ets

3.3.5.2 文件源代码

```
1. //该组件为导航栏，除注册登录页面外，其他页面均需要引用该组件
2. import router from '@ohos.router';
3. @Component
4. export struct navigationBar{
5.   @State title: string | Resource = $r('app.string.default');
6.   @State isBack: boolean = true;
7.   @State isMenu: boolean = true;
8.   @State isMain: boolean = true;
9.   @State isMore: boolean = false;
10.  @State isClose: boolean = false;
11.  @State bgColor: Resource = $r('app.color.light_blue2');
12.  build(){
13.    Row(){
14.      if(this.isMain == true){
15.        Column(){
16.          Image($r('app.media.qq0'))//头像
17.            .width(40)
18.            .height(40)
19.            .borderRadius(100)
20.            .onClick(() => {
21.              router.pushUrl({ url: 'pages/Myself' })
22.            })
23.        }.width(30).padding({left: 30}).onClick(() => {
24.          router.pushUrl({ url: 'pages/Myself' })
25.        })
26.      }
27.      if(this.isBack == true){
28.        Column(){
29.          Image($r('app.media.ic_back'))
30.            .width(26)
31.            .height(26)
32.            .onClick(() => {
33.              router.back()
34.            })
35.        }.width(50)
36.      }
37.      Column(){
38.        Text(this.title )
39.          .fontSize(18)
40.          .fontFamily($r('sys.float.ohos_id_text_size_sub_title2'))
```

```

41.         .maxLines(1)
42.         .textOverflow({ overflow: TextOverflow.Ellipsis })
43.         .fontColor($r('app.color.white'))
44.     }.layoutWeight(1).padding({left: 20})
45.     Column(){
46.         if(this.isMenu == true){
47.             Image($r('app.media.ic_add'))
48.                 .width(40)
49.                 .height(40)
50.         }
51.         if(this.isMore == true){
52.             Image($r('app.media.ic_more'))
53.                 .width(26)
54.                 .height(26)
55.         }
56.     }.width(50)
57. }
58. .height(60)
59. .width('100%')
60. .backgroundColor(this.bgColor)
61. }
62. }

```

3.3.5.3 布局结构

导航栏模块主要采用 Row 容器，将页面元素横向排列。通过使用 Column 组件对每个功能块（如返回按钮、标题、头像、菜单等）进行垂直布局，使得页面结构清晰且易于维护。布局中包含了多个条件渲染的部分，例如根据不同页面需求显示或隐藏某些元素（如返回按钮、菜单按钮等），从而实现了一个灵活的导航栏设计。

3.3.5.4 按钮设计

- 头像按钮：如果用户位于主页面，则显示一个头像按钮，点击该按钮可以跳转到个人中心页面。
- 返回按钮：当需要返回功能时，头像处则显示返回按钮，点击后调用 `router.back()` 函数实现页面返回。
- 菜单按钮和更多按钮：通过条件渲染显示或隐藏不同的操作按钮，如“添加”或“更多”图标，进一步增强了导航栏的功能性。

3.3.5.5 关键函数

通过多个 if 语句，根据页面的不同需求动态渲染导航栏中的元素。

@State title：用于动态设置导航栏的标题内容，支持字符串或资源文件内容。不同页面可以通过修改 title 显示不同的页面标题。

@State isBack：控制是否显示返回按钮。通常在需要提供返回功能的页面中将其设置为 true，否则为 false。

@State isMenu：决定是否显示菜单按钮。如果页面有额外操作，如添加功能，则 isMenu 为 true。

@State isMain：控制是否显示用户头像。如果当前页面是主页面，则设置为 true，显示头像。



@State isMore: 用于显示更多操作按钮,如更多选项菜单。根据页面需求,动态控制显示与否。

@State isClose: 用于控制某些页面是否显示关闭按钮。

@State bgColor: 控制导航栏的背景颜色。

3.3.5.6 页面功能

导航栏模块提供了统一的顶部导航功能,能够根据页面的具体需求灵活调整显示内容。支持设置页面标题、返回功能、头像导航、菜单操作等,提升了页面的交互性和可操作性。通过@State 成员变量的动态控制,导航栏可以适应不同页面场景,为不同页面提供一致且灵活的用户体验。

3.3.6 主页面内消息 TAB 组件实现

3.3.6.1 涉及的文件

src > main > ets > common > components > tabcontent > SessionTabContent.ets

src > main > ets > common > components > navigationBar.ets

src > main > ets > common > components > listitem > SessionListItem.ets

src > main > ets > model > data > SessionData.ets

src > main > ets > model > SessionModel.ets

3.3.6.2 文件源代码

SessionTabContent.ets 代码:

```
1. //消息 TAB 页
2. import { SessionData } from '../../../model/data/SessionData'
3. import { getSessionDatas } from '../../../model/SessionModel'
4. import { SessionListItem } from '../../../components/listitem/SessionListItem'
5. import { navigationBar } from '../../../components/navigationBar'
6. @Component
7. export struct SessionTabContent {
8.     private sessionItem: SessionData[] = getSessionDatas()
9.
10.     build() {
11.         Flex({ direction: FlexDirection.Column, alignItems: ItemAlign.Start, justifyContent: FlexAlign.Start }) {
12.             navigationBar({
13.                 title: '消息',
14.                 isBack: false,
15.                 isClose: false,
16.                 isMenu: true,
17.                 isMain: true,
18.                 isMore: false
19.             })
20.             Column() {
21.                 List() {
22.                     ForEach(this.sessionItem, (item: SessionData) => {
23.                         ListItem() {
24.                             SessionListItem({ sessionItem: item })
```

```

25.         }
26.         }, (item: SessionData) => item.qqid)
27.     }
28. }
29.     .width('100%').height('100%')
30.     .backgroundColor($r('app.color.white'))
31. }
32. }
33. }

```

SessionListItem.ets 代码:

```

1. // 该组件为消息 TAB 的 item 布局
2. import { SessionData } from '../../model/data/SessionData'
3. import router from '@ohos.router';
4. @Component
5. export struct SessionListItem{
6.     private sessionItem: SessionData = new SessionData(
7.         'default_qqid',
8.         'default_headImg',
9.         'default_name',
10.        'default_lastMsg',
11.        0
12.    )
13.    @State itemBgColor: Resource = $r('app.color.white')
14.    build(){
15.        Row(){
16.            //头像
17.            Flex({ direction: FlexDirection.Row , justifyContent: FlexAlign.Center , align
nItems:
18.                ItemAlign.Center}){
19.                Image($rawfile(this.sessionItem.headImg))
20.                .width('100%')
21.                .height('100%')
22.                .borderRadius('100.00vp')
23.                .width('70%').height('70%')
24.            }.width(78).height(78)
25.            //昵称
26.            Column(){
27.                Flex({ direction: FlexDirection.Column , justifyContent: FlexAlign.Center})
28.                {
29.                    Text(this.sessionItem.name)
30.                    .width('100%')
31.                    .textOverflow({ overflow: TextOverflow.Ellipsis })
32.                    .maxLines(1)
33.                    .fontFamily($r('sys.float.ohos_id_text_size_sub_title3'))

```

```

33.         .fontSize(20)
34.         .fontColor($r('app.color.black'))
35.         .margin({ bottom: 1 })
36.         Text(this.sessionItem.lastMsg)
37.         .width('100%')
38.         .textOverflow({ overflow: TextOverflow.Ellipsis })
39.         .maxLines(1)
40.         .fontFamily($r('sys.float.ohos_id_text_size_body1'))
41.         .fontSize(16)
42.         .fontColor($r('app.color.grey3'))
43.         .margin({ top: 1 })
44.     }.width('100%').height(77)
45.     Divider()
46.     .width('100%')
47.     .color($r('app.color.grey2'))
48.     .strokeWidth(0.8)
49.     }.height(78).layoutWeight(1)
50. }
51. .height(78)
52. .width('100%')
53. .backgroundColor(this.itemBgColor)
54. .onTouch((event: TouchEvent) => {
55.     if (event.type === TouchType.Down) {
56.         this.itemBgColor = $r('app.color.grey2')
57.     }
58.     if (event.type === TouchType.Up) {
59.         this.itemBgColor = $r('app.color.white')
60.     }
61. })
62. .onClick(() => {
63.     router.pushUrl({ url: 'pages/Message' ,
64.         params: { sessionId: this.sessionItem }})
65. })
66. }
67. }

```

SessionData.ets 代码:

```

1. //消息列表的数据结构
2. export class SessionData{
3.     qqid: string        //用户 qqid
4.     headImg: string     //头像
5.     name: string        //用户昵称
6.     lastMsg: string      //最后一条消息简要内容
7.     unreadMsg: number    //未读消息数量

```

```

8.   constructor(qqid: string , headImg: string , name: string , lastMsg: string , unR
    eadMsg: number){
9.     this.qqid = qqid
10.    this.headImg = headImg
11.    this.name = name
12.    this.lastMsg = lastMsg
13.    this.unreadMsg = unreadMsg
14.  }
15. }

```

SessionModel.ets 代码:

```

1.  //消息列表的数据样例
2.  import { SessionData } from './data/SessionData'
3.  export function getSessionDatas(): Array<SessionData> {
4.    let sessionsDataArray: Array<SessionData> = []
5.    SessionsComposition.forEach((item: SessionItem) => {
6.      sessionsDataArray.push(new SessionData(item.qqid , item.headImg , item.name , i
        tem.lastMsg , item.unreadmsg))
7.    })
8.    return sessionsDataArray;
9.  }
10. //消息列表数据测试样例
11. interface SessionItem {
12.   qqid: string;
13.   headImg: string;
14.   name: string;
15.   lastMsg: string;
16.   unreadmsg: number;
17. }
18. const SessionsComposition: SessionItem[] = [
19.   {
20.     qqid: "2830133296",
21.     headImg: "qq1.jpg",
22.     name: "非门控通道",
23.     lastMsg: "[语音][在吗]",
24.     unreadmsg: 0
25.   },
26.   {
27.     qqid: "1685733707",
28.     headImg: "qq2.jpg",
29.     name: "欧几里得",
30.     lastMsg: "你好",
31.     unreadmsg: 0
32.   },
33.   {

```

```

34.     qqid: "1",
35.     headImg: "mypc.jpg",
36.     name: "我的电脑",
37.     lastMsg: "[文件]1120200325.pdf",
38.     unreadMsg: 0
39.   }
40. ]

```

3.3.6.3 布局结构

- SessionTabContent.ets:

该文件定义了消息标签页的布局，使用 Flex 容器来组织内容，布局方向为纵向（FlexDirection.Column），从上到下排列。顶部使用 navigationBar 组件展示标题和导航功能，主体部分通过 Column 容器展示消息列表（List()），每条消息项由 SessionListItem 组件渲染。

- SessionListItem.ets:

消息项布局文件 SessionListItem.ets，通过 Row 容器进行横向布局，包括头像、昵称、最后一条消息等内容。头像使用 Image() 组件渲染，文本信息则由 Text() 组件和 Flex 容器垂直排列。消息项的背景颜色会根据用户的触摸事件动态改变，提供了良好的视觉反馈。

3.3.6.4 按钮设计

- SessionListItem.ets:

每个消息项本质上是一个按钮，点击后会跳转到相应的消息详情页面。此按钮不仅响应点击事件，还能够根据触摸事件改变背景颜色，增强用户的交互体验。

3.3.6.5 关键函数

- SessionTabContent.ets:

getSessionDatas(): 该函数从 SessionModel.ets 文件中引入，负责获取会话数据并展示在消息列表中。通过调用该函数，消息标签页可以动态加载当前的会话信息。

- SessionListItem.ets:

router.pushUrl({ url: 'pages/Message', params: { sessionData: this.sessionItem } }): 此函数在消息项点击时触发，用于页面跳转到具体的消息详情页。传递的参数包含用户的昵称、账号等信息，确保用户点击某一消息项后能正确查看到对应的聊天记录。

- SessionData.ets & SessionModel.ets:

SessionData 类：该类定义了每个消息项的数据结构，包括 qqid、headImg（头像）、name（昵称）、lastMsg（最后一条消息）等属性。

getSessionDatas(): 通过从消息列表样例数据中生成 SessionData 对象，该函数负责返回当前的会话数据，用于在消息标签页中展示。

3.3.6.6 页面功能

- SessionTabContent.ets:

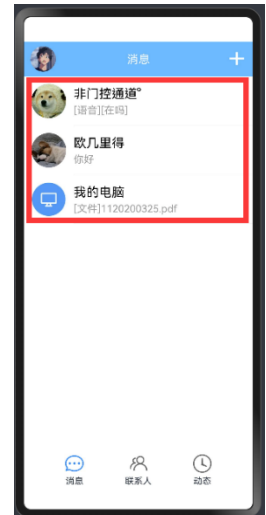
该文件实现了消息标签页的主体功能，加载并展示当前所有会话的信息。用户可以通过此页面查看最近的消息列表，点击某条消息进入具体的会话详情页面。

- SessionListItem.ets:

SessionListItem 文件实现了每条消息项的布局与功能，包括头像、昵称、最后一条消息内容的展示。用户可以通过点击消息项进入对应的会话详情页面，同时在触摸时提供视觉反馈，提升交互体验。

- SessionData.ets & SessionModel.ets:

这些文件负责定义会话数据的结构与样例数据，确保在消息标签页中可以正确加载并



展示会话信息。

3.3.7 optionItem 页面实现

3.3.7.1 涉及的文件

src > main > ets > common > utils > optionItem.ets

3.3.7.2 文件源代码

```
1. //该组件为设置和各功能选项的布局
2. @Component
3. export struct optionItem {
4.     @State bgColor: Resource = $r("app.color.white")
5.     private img: Resource = $r('app.media.default') //图标资源
6.     private context: Resource = $r('app.string.default') //文字资源
7.     private isNext: boolean = false //是否有下一步的导航箭头
8.     private marginTop: number = 0 //组件上边距
9.     private itemOnClick = (event: ClickEvent) =>{} //item 的单击事件（可选）
10.    private usePic: number = 1 //是否使用图片
11.    private isDown: boolean = false //是否有下一步的导航箭头
12.    build(){
13.        Row(){
14.            if(this.usePic == 1){
15.                Flex({direction: FlexDirection.Row , alignItems:ItemAlign.Center , justifyC
ontent:FlexAlign.Center}){
16.                    Image(this.img)
17.                        .width(30).height(30)
18.                }.width(50).height(60)
19.            }
20.            Column(){
21.                Row(){
22.                    Text(this.context)
23.                        .fontFamily($r('sys.float.ohos_id_text_size_body1'))
24.                        .fontSize(19)
25.                        .textOverflow({ overflow: TextOverflow.Ellipsis })
26.                        .maxLines(1)
27.                        .layoutWeight(1)
28.                    if(this.isNext == true){
29.                        Image($r('app.media.ic_next'))
30.                            .width(18)
31.                            .height(18)
32.                            .margin(10)
33.                    }
34.                    if(this.isDown == true){
35.                        Image($r('app.media.ic_down'))
36.                            .width(18)
37.                            .height(18)
```



```

38.         .margin(10)
39.     }
40.     }.height(59)
41. }
42. .padding({left: 10 , right:4})
43. .height(60)
44. .layoutWeight(1)
45. }.width('100%').height(60)
46. .margin({top: this.marginTop})
47. .backgroundColor(this.bgColor)
48. .onTouch((event: TouchEvent) => {
49.     if (event.type === TouchType.Down) {
50.         this.bgColor = $r('app.color.grey2')
51.     }
52.     if (event.type === TouchType.Up) {
53.         this.bgColor = $r('app.color.white')
54.     }
55. })
56. .onClick((event: ClickEvent) => {
57.     this.itemOnClick(event)
58. })
59. }
60. }

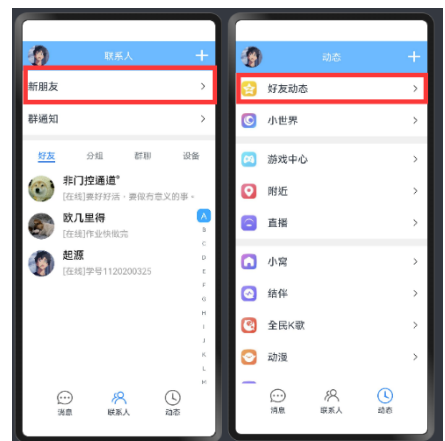
```

3.3.7.3 布局结构

该组件使用 Row 作为主容器，将图标和文字横向排列。图标位于左侧，文字位于右侧，并且如果需要，还会在文字旁边显示一个导航箭头或下拉箭头。

图标部分通过 Image() 组件渲染，宽高设置为 30 x 30。文字部分使用 Text() 组件，并通过 Ellipsis 实现文字过长时的省略效果。此外，文字部分还支持动态设置字体大小和样式，确保统一的视觉效果。

布局结构考虑了动态调整的需求，例如可以通过 usePic 来控制是否显示图标，通过 isNext 和 isDown 来控制是否显示导航箭头。



3.3.7.4 按钮设计

• 点击反馈：

组件集成了点击事件的反馈逻辑，当用户按下时，背景颜色会变为灰色，松开时恢复为白色。

• 单击事件：

通过自定义的点击事件处理器 (itemOnClick)。可以向下一个页面传入用户数据。

• 导航箭头：

通过 isNext 和 isDown 两个布尔值控制，组件能够根据需要显示导航箭头或者下拉箭头。

3.3.7.5 关键函数

onTouch() 事件：用于处理触摸事件。根据触摸类型 (TouchType.Down 和

TouchType.Up), 动态改变背景颜色。

onClick() 事件: 处理组件的点击事件, 执行传入的自定义逻辑。该功能可以为每个选项绑定不同的操作, 比如跳转页面或显示更多选项。

条件渲染: 通过 if 判断语句来动态渲染组件的不同部分, 例如是否显示导航箭头或下拉箭头。

3.3.7.6 页面功能

该组件设计为通用选项条目, 适用于设置页面、功能选项等场景。无论是带图标的选项、简单的文字选项, 还是需要导航的选项, 都可以通过设置不同的状态变量实现。

页面可以通过布尔值控制是否显示图标、导航箭头或下拉箭头, 并通过 marginTop 动态调整组件的边距, 适应不同的页面布局需求。还可以根据实际需求传递自定义的点击事件处理逻辑。

3.3.8 主页面内联系人 TAB 组件实现

3.3.8.1 涉及的文件

src > main > ets > common > components > tabcontent > ContactTabContent.ets

src > main > ets > common > components > navigationBar.ets

src > main > ets > common > components > listitem > ContactListItem.ets

src > main > ets > common > utils > optionItem.ets

src > main > ets > model > data > ContactData.ets

src > main > ets > model > ContactModel.ets

3.3.8.2 文件源代码

ContactTabContent.ets 代码:

```
1. //联系人 TAB 页
2. import { ContactData, Gender } from '../../model/data/ContactData'
3. import { getContactDatas } from '../../model/ContactModel'
4. import { ContactListItem } from '../listitem/ContactListItem'
5. import { navigationBar } from '../../components/navigationBar'
6. import { optionItem } from '../../utils/optionItem'
7. interface SystemItem {
8.     qqid: string;
9.     headImg: string;
10.    name: string;
11.    mark: string;
12.    gender: Gender;
13.    area: string;
14. }
15. @Component
16. export struct ContactTabContent {
17.    private value: string[] = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K',
        'L', 'M', 'N', 'O']
18.    private contactItems: ContactData[] = getContactDatas() //获取联系人数据
19.    private scroller: Scroller = new Scroller()
20.    build() {
```

```

21.     Flex({ direction: FlexDirection.Column, alignItems: ItemAlign.Start, justifyCon
tent: FlexAlign.Start }) {
22.         navigationBar({
23.             title: '联系人',
24.             isBack: false,
25.             isClose: false,
26.             isMenu: true,
27.             isMain: true,
28.             isMore: false
29.         })
30.         Stack() {
31.             Column() {
32.                 Column() {
33.                     optionItem({
34.                         context: $r('app.string.info_btn_nf'),
35.                         isNext: true,
36.                         marginTop: 0,
37.                         usePic: 0
38.                     })
39.                     Divider()
40.                         .width('100%')
41.                         .color($r('app.color.grey2'))
42.                         .strokeWidth(0.8)
43.                     optionItem({
44.                         context: $r('app.string.info_btn_qtz'),
45.                         isNext: true,
46.                         marginTop: 0,
47.                         usePic: 0
48.                     })
49.                 }
50.                 Tabs() {
51.                     TabContent() {
52.                         Stack() {
53.
54.                             Column() {
55.                                 ForEach(this.contactItems, (item: ContactData) => {
56.                                     ListItem() {
57.                                         ContactListItem({ contactItem: item })
58.                                     }
59.                                 }, (item: ContactData) => item.qqid)
60.                             }.height('100%')
61.                             Flex({ direction: FlexDirection.Row, justifyContent: FlexAlign.End,
alignItems: ItemAlign.Center }) {
62.                                 AlphabetIndexer({ arrayValue: this.value, selected: 0 })
63.                                     .selectedColor($r('app.color.white'))// 选中颜色
64.

```

```

65.
66.         .selectedBackgroundColor($r('app.color.light_blue3'))// 选中背景
    颜色
67.         .popupBackground($r('app.color.grey3'))// 弹出框背景颜色
68.
69.         .selectedFont({ size: 16 })// 选中的样式
70.
71.         .itemSize(26)// 每一项的大小正方形
72.         .alignStyle(IndexerAlign.Right)// 左对齐
73.         .padding({ right: 8 })
74.
75.     }.width('100%')
76. }
77. }
78. .tabBar('好友')
79. TabContent() {
80.     Column() {
81.         optionItem({
82.             context: $r('app.string.info_btn_cm'),
83.             isDown: true,
84.             marginTop: 0,
85.             usePic: 0
86.         })
87.         Divider()
88.             .width('100%')
89.             .color($r('app.color.grey2'))
90.             .strokeWidth(0.8)
91.         ForEach(this.contactItems, (item: ContactData) => {
92.             ListItem() {
93.                 ContactListItem({ contactItem: item })
94.             }
95.         }, (item: ContactData) => item.qqid)
96.     }.height('100%')
97. }
98. .tabBar('分组')
99. TabContent() {
100.     Text('暂无群聊').fontSize(30)
101. }
102. .tabBar('群聊')
103. TabContent() {
104.     Column() {
105.         ContactListItem({
106.             contactItem: new ContactData(
107.                 "1",
108.                 "mypc.jpg",
109.                 "我的电脑",

```

```

110.         "我的电脑",
111.         Gender.male,
112.         "电脑",
113.         "无需数据线, 手机轻松传文件到电脑")
114.     })
115.     ContactListItem({
116.         contactItem: new ContactData(
117.             "2",
118.             "fnd.jpg",
119.             "发现新设备",
120.             "发现新设备",
121.             Gender.male,
122.             "新设备",
123.             "搜索附近的设备, 用 QQ 轻松连接设备")
124.     })
125.     }.height('100%')
126.     }
127.     .tabBar("设备")
128.     }.backgroundColor($r('app.color.white'))
129.     .width('100%').height('100%').margin({ top: 10 })
130.     }
131.     .width('100%').height('100%')
132.     .backgroundColor($r('app.color.grey2'))
133.
134.     }
135.     }
136.     }
137. }

```

ContactListItem.ets 代码:

```

1. //联系人 TAB 页面的 item 布局
2. import { ContactData, Gender } from '../../model/data/ContactData'
3. import router from '@ohos.router';
4. @Component
5. export struct ContactListItem {
6.     private contactItem: ContactData = new ContactData(
7.         'default_qqid',
8.         'default_headImg',
9.         'default_name',
10.        'default_mark',
11.        Gender.male,
12.        'default_area',
13.        'default_signature'
14.    )
15.    @State itemBgColor: Resource = $r('app.color.white')

```

```

16.   build(){
17.       Row(){
18.           //头像
19.           Flex({ direction: FlexDirection.Row , justifyContent: FlexAlign.Center , align
nItems:
20.               ItemAlign.Center}){
21.               Image($rawfile(this.contactItem.headImg))
22.               .width('100%')
23.               .height('100%')
24.               .borderRadius(100)
25.               .width('70%').height('70%')
26.           }.width(70).height(70)
27.           //昵称或备注
28.           Flex({ direction: FlexDirection.Column}){
29.
30.               Text(this.contactItem.mark)
31.               .textOverflow({ overflow: TextOverflow.Ellipsis })
32.               .maxLines(1)
33.               .fontFamily($r('sys.float.ohos_id_text_size_sub_title3'))
34.               .fontSize(20)
35.               .fontColor($r('app.color.black'))
36.               .margin({ top: 6 ,bottom: 5 })
37.               Text(this.contactItem.signature)
38.               .width('100%')
39.               .textOverflow({ overflow: TextOverflow.Ellipsis })
40.               .maxLines(1)
41.               .fontFamily($r('sys.float.ohos_id_text_size_body1'))
42.               .fontSize(16)
43.               .fontColor($r('app.color.grey3'))
44.               .margin({ top: 1 })
45.           }.height(70).layoutWeight(1).padding({left: 5})
46.       }
47.       .height(70)
48.       .width('100%')
49.       .backgroundColor(this.itemBgColor)
50.       .onTouch((event: TouchEvent) => {
51.           if (event.type === TouchType.Down) {
52.               this.itemBgColor = $r('app.color.grey2')
53.           }
54.           if (event.type === TouchType.Up) {
55.               this.itemBgColor = $r('app.color.white')
56.           }
57.       })
58.       .onClick(() => {
59.           router.pushUrl({ url: 'pages/UserInfo' ,
60.               params: { contactData: this.contactItem }})

```

```
61.    })
62.  }
63. }
```

ContactData.ets 代码:

```
1.  //联系人 TAB 的联系人数据结构
2.  export enum Gender{
3.    //女性
4.    female ,
5.    //男性
6.    male
7.  }
8.  export class ContactData{
9.    qqid: string      //用户 qqid
10.   headImg: string   //头像
11.   name: string      //用户昵称
12.   mark: string      //备注名
13.   gender: Gender    //性别
14.   area: string      //地区
15.   signature: string //个性签名
16.   constructor(qqid: string , headImg: string , name: string , mark: string , gender
    : Gender, area: string, signature: string){
17.     this.qqid = qqid
18.     this.headImg = headImg
19.     this.name = name
20.     this.mark = mark
21.     this.gender = gender
22.     this.area = area
23.     this.signature = signature
24.   }
25. }
```

ContactModel.ets 代码:

```
1.  //联系人 TAB 的联系人数据
2.  import { ContactData , Gender } from './data/ContactData'
3.  interface ContactItem {
4.    qqid: string;
5.    headImg: string;
6.    name: string;
7.    mark: string;
8.    gender: Gender;
9.    area: string;
10.   signature: string;
11. }
```

```

12. export function getContactDatas(): Array<ContactData> {
13.   let contactDataArray: Array<ContactData> = []
14.   ContactsComposition.forEach((item: ContactItem) => {
15.     contactDataArray.push(new ContactData(item.qqid , item.headImg , item.name , item.mark , item.gender , item.area, item.signature))
16.   })
17.   return contactDataArray;
18. }
19. const ContactsComposition: ContactItem[] = [
20.   {
21.     qqid: "2830133296" ,
22.     headImg: "qq1.jpg" ,
23.     name: "非门控通道" ,
24.     mark: "非门控通道",
25.     gender: Gender.male ,
26.     area: "河北·保定",
27.     signature: "[在线]要好好活, 要做有意义的事。"
28.   },
29.   {
30.     qqid: "1685733707" ,
31.     headImg: "qq2.jpg" ,
32.     name: "欧几里得" ,
33.     mark: "欧几里得",
34.     gender: Gender.male ,
35.     area: "湖北·武汉",
36.     signature: "[在线]作业快做完"
37.   },
38.   {
39.     qqid: "1627018151" ,
40.     headImg: "qq0.jpg" ,
41.     name: "张泽渊" ,
42.     mark: "起源",
43.     gender: Gender.male ,
44.     area: "北京·海淀",
45.     signature: "[在线]学号 1120200325"
46.   }
47. ]

```


3.3.8.3 布局结构

- ContactTabContent.ets:

整个联系人页面使用了 Flex 容器进行纵向布局 (FlexDirection.Column), 页面从上至下排列, 顶部为 navigationBar 导航栏组件。

页面主体部分包含了两个选项条目 (使用 optionItem 组件) 和一个 Tabs 组件。Tabs 组件内有多个选项卡, 每个选项卡包含不同的联系人信息, 分为“好友”、“分组”、“群聊”和“设备”四类内容。

联系人列表通过 ForEach 循环遍历联系人数据, 并使用 ContactListItem 组件进行展示。

- ContactListItem.ets:

每个联系人条目使用了 Row 容器进行横向布局, 包含头像、昵称或备注、签名等信息。

3.3.8.4 按钮设计

- ContactTabContent.ets:

顶部导航栏有多个功能按钮, 通过 navigationBar 组件实现, 比如返回、菜单等按钮, 具体显示哪些按钮通过布尔变量控制。

在联系人列表中, 每个联系人条目本质上都是一个按钮。点击这些条目可以跳转到相应的用户信息页面, 这通过 ContactListItem 组件中的点击事件实现。

- ContactListItem.ets:

头像部分和文字部分都是可点击的, 触发点击事件时, 会跳转到用户详情页面。整个条目还设计了触摸反馈, 在用户按下和松开时, 背景颜色会相应变化。

3.3.8.5 关键函数

- getContactDatas():

这个函数位于 ContactModel.ets 中, 用于从静态数据源获取联系人数据, 并返回一个包含 ContactData 对象的数组。这是联系人列表的核心数据来源。

- router.pushUrl():

在 ContactListItem.ets 中, 当用户点击某个联系人条目时, 会调用 router.pushUrl() 函数, 跳转到用户信息页面, 同时传递当前联系人数据作为参数。

- AlphabetIndexer():

模拟在联系人页面中实现快速定位到某个字母开头的联系人的功能。这个组件模拟用户快速选择某个字母, 并跳转到相应的联系人位置。

3.3.8.6 页面功能

页面实现了联系人标签页的整体布局, 能够显示联系人列表, 并允许用户在不同选项卡之间切换。

通过 optionItem 组件, 页面还可以展示设置或功能选项, 并通过点击进行操作。

ContactListItem 组件负责展示单个联系人条目, 包含头像、昵称、签名等内容, 并提供点击交互和触摸反馈。当用户点击某个联系人时, 会跳转到该联系人的详细信息页面。

通过使用 ForEach 循环, 可以动态渲染多个联系人条目, 这使得联系人列表可以根据实际数据动态生成。

ContactModel.ets 中的样例数据为前端提供了静态联系人列表, 这些数据可以被前端页面读取并展示。所有联系人信息都通过 ContactData 类进行管理, 确保数据结构一致。



3.3.9 主页面内动态 TAB 组件实现

3.3.9.1 涉及的文件

src > main > ets > common > components > tabcontent > DiscoveryTabContent.ets

src > main > ets > common > components > navigationBar.ets

src > main > ets > common > utils > optionItem.ets

3.3.9.2 文件源代码

```
1. //动态 TAB
2. import { optionItem } from '../utils/optionItem'
3. import { navigationBar } from '../components/navigationBar'
4. import router from '@ohos.router';
5. @Component
6. export struct DiscoveryTabContent {
7.     private scroller: Scroller = new Scroller()
8.     private momentItemOnClick(event: ClickEvent) {
9.         router.pushUrl({ url: 'pages/Moments' })
10.    }
11.    build(){
12.        Column(){
13.            navigationBar({
14.                title: '动态',
15.                isBack: false,
16.                isClose: false,
17.                isMenu: true,
18.                isMain: true,
19.                isMore: false
20.            })
21.            Scroll(this.scroller) {
22.                Column() {
23.                    optionItem({
24.                        img: $r('app.media.dt1'),
25.                        context: $r('app.string.dt1'),
26.                        isNext: true,
27.                        marginTop: 0,
28.                        itemOnClick: this.momentItemOnClick
29.                    })
30.                    optionItem({
31.                        img: $r('app.media.dt2'),
32.                        context: $r('app.string.dt2'),
33.                        isNext: true,
34.                        marginTop: 0
35.                    })
36.                    optionItem({
37.                        img: $r('app.media.dt3'),
38.                        context: $r('app.string.dt3'),
39.                        isNext: true,
40.                        marginTop: 13
41.                    })

```

```

42.         optionItem({
43.             img: $r('app.media.dt4'),
44.             context: $r('app.string.dt4'),
45.             isNext: true,
46.             marginTop: 0
47.         })
48.         optionItem({
49.             img: $r('app.media.dt5'),
50.             context: $r('app.string.dt5'),
51.             isNext: true,
52.             marginTop: 0
53.         })
54.         optionItem({
55.             img: $r('app.media.dt6'),
56.             context: $r('app.string.dt6'),
57.             isNext: true,
58.             marginTop: 13
59.         })
60.         optionItem({
61.             img: $r('app.media.dt7'),
62.             context: $r('app.string.dt7'),
63.             isNext: true,
64.             marginTop: 0
65.         })
66.         optionItem({
67.             img: $r('app.media.dt8'),
68.             context: $r('app.string.dt8'),
69.             isNext: true,
70.             marginTop: 0
71.         })
72.         optionItem({
73.             img: $r('app.media.dt9'),
74.             context: $r('app.string.dt9'),
75.             isNext: true,
76.             marginTop: 0
77.         })
78.         optionItem({
79.             img: $r('app.media.dt10'),
80.             context: $r('app.string.dt10'),
81.             isNext: true,
82.             marginTop: 0
83.         })
84.         optionItem({
85.             img: $r('app.media.dt11'),
86.             context: $r('app.string.dt11'),
87.             isNext: true,

```

```

88.         marginTop: 0
89.     })
90.     optionItem({
91.         img: $r('app.media.dt12'),
92.         context: $r('app.string.dt12'),
93.         isNext: true,
94.         marginTop: 0
95.     })
96.     optionItem({
97.         img: $r('app.media.dt13'),
98.         context: $r('app.string.dt13'),
99.         isNext: true,
100.        marginTop: 0
101.    })
102.
103.    optionItem({
104.        img: $r('app.media.dt14'),
105.        context: $r('app.string.dt14'),
106.        isNext: true,
107.        marginTop: 0
108.    })
109.
110.    optionItem({
111.        img: $r('app.media.dt15'),
112.        context: $r('app.string.dt15'),
113.        isNext: true,
114.        marginTop: 0
115.    })
116.
117.    optionItem({
118.        img: $r('app.media.dt16'),
119.        context: $r('app.string.dt16'),
120.        isNext: true,
121.        marginTop: 0
122.    })
123.
124.    optionItem({
125.        img: $r('app.media.dt17'),
126.        context: $r('app.string.dt17'),
127.        isNext: true,
128.        marginTop: 0
129.    })
130.
131.    optionItem({
132.        img: $r('app.media.dt18'),
133.        context: $r('app.string.dt18'),

```

```

134.         isNext: true,
135.         marginTop: 0
136.     })
137.     optionItem({
138.         img: $r('app.media.dt19'),
139.         context: $r('app.string.dt19'),
140.         isNext: true,
141.         marginTop: 0
142.     })
143.     }
144.     .width('100%')
145.     }
146.     .scrollable(ScrollDirection.Vertical)
147.     .scrollBar(BarState.Off)
148.     }
149.     .width('100%').height('100%')
150.     .backgroundColor($r('app.color.grey2'))
151.     }
152. }

```

3.3.9.3 布局结构

页面主要采用了 Column 容器实现纵向布局，内容从上至下排列。顶部包含一个 navigationBar 组件，显示页面的标题“动态”以及相应的导航按钮。

内容部分包裹在 Scroll 组件中，允许页面在垂直方向滚动，增强用户体验。

页面主体部分由一系列的 optionItem 组件构成，这些组件依次展示各种动态内容。通过 Column 容器将这些选项条目整齐排列，并通过不同的 marginTop 值控制各个条目之间的间距，使布局更具层次感。

3.3.9.4 按钮设计

每个动态项都使用了 optionItem 组件，其中“好友动态”optionItem 组件绑定了点击事件，用户点击时可以跳转到空间动态页面。

3.3.9.5 关键函数

momentItemOnClick 函数：

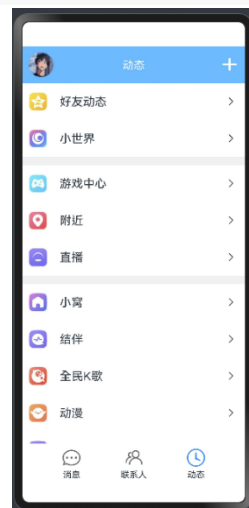
这是一个自定义点击事件处理函数，当用户点击第一个动态项时，router.pushUrl() 函数会被调用，页面将跳转到空间动态页面。

3.3.9.6 页面功能

页面的主要功能是展示一系列动态选项，用户可以浏览不同的选项内容。通过使用 optionItem 组件可以展示图文结合的选项，同时通过导航箭头引导用户进一步操作。

页面内容较多，无法一次性完全显示在屏幕上，因此通过 Scroll 组件实现了内容的垂直滚动功能，让用户可以轻松浏览所有动态项。加上每个 optionItem 的导航功能，用户点击后可以进入详细页面或执行相应操作。

通过这种结构，页面可以很方便地扩展更多的动态项或功能。只需在 Column 容器内增加新的 optionItem 组件，并为其定义点击事件或跳转逻辑即可，具备良好的灵活性。



3.3.10 个人页面实现

3.3.10.1 涉及的文件

src > main > ets > pages > Myself.ets

src > main > ets > common > utils > optionItem.ets

3.3.10.2 文件源代码

```
1. //个人信息界面
2. import { optionItem } from '../common/utils/optionItem'
3. import router from '@ohos.router';
4. @Entry
5. @Component
6. struct Myself {
7.   private headerImg: string = 'qq0.jpg'
8.   private name: string = '张泽渊'
9.   private userId: string = '1627018151'
10.  private scrollerForScroll: Scroller = new Scroller()
11.  private settingItemOnClick(event: ClickEvent) {
12.    router.pushUrl({ url: 'pages/Setting' })
13.  }
14.  build() {
15.    Column(){
16.      Stack({}) {
17.        Image($r('app.media.gradualchange'))
18.          .width('100%')
19.        Column() {
20.          Row() {
21.            Image($r('app.media.dk'))
22.              .width(26)
23.              .height(26)
24.              .margin({right: 10})
25.            Text('今天还没打卡哦')
26.              .fontColor($r('app.color.white')).margin({right: 150})
27.            Image($r('app.media.ic_close'))
28.              .width(26)
29.              .height(26)
30.              .onClick(() => {
31.                router.back()
32.              })
33.          }.margin({top: 20})
34.          Row() {
35.            Image($rawfile(this.headerImg))
36.              .width(80)
37.              .height(80)
38.              .borderRadius(100)
39.              .margin(20)
```

```

39.         Column() {
40.             Text(this.name)
41.                 .fontFamily($r('sys.float.ohos_id_text_size_sub_title1'))
42.                 .fontSize(28)
43.                 .textOverflow({ overflow: TextOverflow.Clip })
44.                 .maxLines(1)
45.                 .fontWeight(FontWeight.Bold)
46.                 .margin({ bottom: 10 })
47.                 .fontColor($r('app.color.white'))
48.             Image($r('app.media.level'))
49.                 .width('80%')
50.             Text() {
51.                 Span('编辑个性签名')
52.                     .fontFamily($r('sys.float.ohos_id_text_size_body1'))
53.                     .fontSize(18)
54.                     .fontColor($r('app.color.white'))
55.             }
56.             .textOverflow({ overflow: TextOverflow.Ellipsis })
57.             .maxLines(1)
58.         }
59.         .alignItems(HorizontalAlign.Start)
60.         .layoutWeight(1)
61.         Image($r('app.media.ic_qr_code'))
62.             .width(30)
63.             .height(30)
64.             .margin({right: 20})
65.     }
66.     .width('100%')
67.     .height(180)
68. }
69. }.width('100%').height('35%')
70. Column() {
71.     List() {
72.         optionItem({
73.             img: $r('app.media.s1'),
74.             context: $r('app.string.me_service1'),
75.             isNext: true,
76.             marginTop: 0
77.         })
78.         optionItem({
79.             img: $r('app.media.s2'),
80.             context: $r('app.string.me_service2'),
81.             isNext: true,
82.             marginTop: 0
83.         })
84.         optionItem({

```

```

85.         img: $r('app.media.s3'),
86.         context: $r('app.string.me_service3'),
87.         isNext: true,
88.         marginTop: 0
89.     })
90.     optionItem({
91.         img: $r('app.media.s4'),
92.         context: $r('app.string.me_service4'),
93.         isNext: true,
94.         marginTop: 0
95.     })
96.     optionItem({
97.         img: $r('app.media.s5'),
98.         context: $r('app.string.me_service5'),
99.         isNext: true,
100.        marginTop: 0
101.    })
102.    optionItem({
103.        img: $r('app.media.s6'),
104.        context: $r('app.string.me_service6'),
105.        isNext: true,
106.        marginTop: 0
107.    })
108.    optionItem({
109.        img: $r('app.media.s7'),
110.        context: $r('app.string.me_service7'),
111.        isNext: true,
112.        marginTop: 0
113.    })
114.    optionItem({
115.        img: $r('app.media.s8'),
116.        context: $r('app.string.me_service8'),
117.        isNext: true,
118.        marginTop: 0
119.    })
120.    optionItem({
121.        img: $r('app.media.s9'),
122.        context: $r('app.string.me_service9'),
123.        isNext: true,
124.        marginTop: 0
125.    })
126.    optionItem({
127.        img: $r('app.media.s10'),
128.        context: $r('app.string.me_service10'),
129.        isNext: true,
130.        marginTop: 0

```



```

131.         })
132.         optionItem({
133.             img: $r('app.media.s11'),
134.             context: $r('app.string.me_service11'),
135.             isNext: true,
136.             marginTop: 0
137.         })
138.     }.height('55%')
139. }
140. Row(){
141.     Column() {
142.         Image($r('app.media.b1'))
143.         .objectFit(ImageFit.Contain)
144.         .width(30).height(30)
145.         Text('设置')
146.         .fontSize(20)
147.         .fontFamily($r('sys.float.ohos_id_text_size_body2'))
148.         .margin({ top: 3 })
149.     }.width('25%')
150.     Column() {
151.         Image($r('app.media.b2'))
152.         .objectFit(ImageFit.Contain)
153.         .width(30).height(30)
154.         Text('夜间')
155.         .fontSize(20)
156.         .fontFamily($r('sys.float.ohos_id_text_size_body2'))
157.         .margin({ top: 3 })
158.     }.width('25%')
159.     Column() {
160.         Image($r('app.media.b3'))
161.         .objectFit(ImageFit.Contain)
162.         .width(30).height(30)
163.         Text('14 天')
164.         .fontSize(20)
165.         .fontFamily($r('sys.float.ohos_id_text_size_body2'))
166.         .margin({ top: 3 })
167.     }.width('25%')
168.     Column() {
169.         Image($r('app.media.b4'))
170.         .objectFit(ImageFit.Contain)
171.         .width(30).height(30)
172.         Text('当地天气')
173.         .fontSize(20)
174.         .fontFamily($r('sys.float.ohos_id_text_size_body2'))
175.         .margin({ top: 3 })
176.     }.width('25%')

```

```

177.         }.height('10%').margin({top: 7}).backgroundColor($r('app.color.white')).onClick(
            ick(this.settingItemOnClick).width('100%')
178.     }
179.     .height('100%')
180.     .backgroundColor($r('app.color.grey2'))
181. }
182. }

```

3.3.10.3 布局结构

页面内容分为三个主要部分：顶部用户信息部分、中间的服务选项列表部分和底部的快捷操作部分。

顶部部分使用了 Stack 和 Row 容器，展示了用户的头像、昵称、个性签名、等级图标等关键信息，并包含了一个返回按钮。

中间部分通过 Column 容器和 List 组件，展示了一系列的服务选项。每个服务选项都使用 optionItem 组件渲染，整齐地排列在页面中间，方便用户进行进一步的操作。

底部部分通过 Row 容器将快捷操作按钮分为四个列，提供了诸如“设置”、“夜间模式”、“连续登录天数”和“当地天气”等功能，用户可以快速访问这些常用功能。

3.3.10.4 按钮设计

服务选项按钮：中间的服务选项列表中的每一项都是一个按钮，使用了 optionItem 组件，并且设置了 isNext: true，即每个选项右边都带有一个导航箭头，提示用户点击这些选项可以跳转到相关页面。

底部快捷操作按钮：底部的快捷操作部分包含了四个主要按钮（如“设置”按钮）。每个按钮由一个图标和文字构成，按钮的点击区域较大，保证了用户在操作时的准确性和便利性。

整个页面的按钮设计均考虑到了用户的交互体验，提供了直观的点击反馈和跳转功能，使用户能够轻松访问各类信息和设置。

3.3.10.5 关键函数

settingItemOnClick():

该函数处理用户点击底部设置按钮后的操作逻辑，具体实现为跳转到设置页面 (Setting)。

3.3.10.6 页面功能

顶部部分的用户信息组件便于用户能够快速查看自己的头像、昵称、个性签名等个人资料，同时也可以快速返回主页面。

中间部分的服务选项列表为用户提供了多个操作入口，提升了应用的功能扩展性。

底部部分的快捷操作按钮设计使得用户可以快速访问常用功能，比如查看天气、设置夜间模式等，增强了界面的实用性和便利性。



3.3.11 设置页面实现

3.3.11.1 涉及的文件

src > main > ets > pages > Setting.ets

src > main > ets > common > components > navigationBar.ets

src > main > ets > common > utils > optionItem.ets

3.3.11.2 文件源代码

```

1. //设置页面
2. import { navigationBar } from '../common/components/navigationBar'
3. import { optionItem } from '../common/utils/optionItem'
4. @Entry
5. @Component
6. struct Setting {
7.     private scroller: Scroller = new Scroller()
8.     build() {
9.         Column(){
10.             navigationBar({title: $r('app.string.me_setting'), isBack:true, isClose:false
, isMenu: false, isMain: false, isMore:false})
11.             Scroll(this.scroller) {
12.                 Column() {
13.                     optionItem({ img:$r('app.media.st1'),context: $r('app.string.set_1'), isN
ext: true, marginTop: 0 })
14.                     optionItem({ img:$r('app.media.st2'),context: $r('app.string.set_2'), isN
ext: true, marginTop: 0 })
15.                     optionItem({ img:$r('app.media.st3'),context: $r('app.string.set_3'), isN
ext: true, marginTop: 0 })
16.                     optionItem({ img:$r('app.media.st4'),context: $r('app.string.set_4'), isN
ext: true, marginTop: 10 })
17.                     optionItem({ img:$r('app.media.st5'),context: $r('app.string.set_5'), isN
ext: true, marginTop: 0 })
18.                     optionItem({ img:$r('app.media.st6'),context: $r('app.string.set_6'), isN
ext: true, marginTop: 0 })
19.                     optionItem({ img:$r('app.media.st7'),context: $r('app.string.set_7'), isN
ext: true, marginTop: 0 })
20.                     optionItem({ img:$r('app.media.st8'),context: $r('app.string.set_8'), isN
ext: true, marginTop: 10 })
21.                     optionItem({ img:$r('app.media.st9'),context: $r('app.string.set_9'), isN
ext: true, marginTop: 10 })
22.                     optionItem({ img:$r('app.media.st10'),context: $r('app.string.set_10'), i
sNext: true, marginTop: 0 })
23.                     optionItem({ img:$r('app.media.st11'),context: $r('app.string.set_11'), i
sNext: true, marginTop: 0 })
24.                     optionItem({ img:$r('app.media.st12'),context: $r('app.string.set_12'), i
sNext: true, marginTop: 0 })
25.                     optionItem({ img:$r('app.media.st13'),context: $r('app.string.set_13'), i
sNext: true, marginTop: 10 })
26.                 }
27.                 .alignItems(HorizontalAlign.Start)
28.             }
29.             .scrollable(ScrollDirection.Vertical)
30.             .scrollBar(BarState.Off)
31.         }

```

```

32.     .width('100%').height('100%')
33.     .backgroundColor($r('app.color.grey2'))
34. }
35. }

```

3.3.11.3 布局结构

设置页面的核心是纵向的选项列表。

顶部导航栏使用了 `navigationBar` 组件，设置了返回按钮（`isBack: true`），方便用户在操作完成后返回上一页面。

页面主体部分使用了 `Scroll` 容器包裹一个 `Column` 容器，确保当设置选项较多时，用户可以通过滚动查看所有内容。页面滚动方向为垂直（`ScrollDirection.Vertical`），且隐藏了滚动条（`scrollBar(BarState.Off)`）。

选项列表部分通过 `optionItem` 组件逐个渲染，每个选项都使用图片和文字进行展示，并且根据需要设置了不同的 `marginTop` 值，来控制选项之间的间距，增强了页面的层次感。

3.3.11.4 按钮设计

导航按钮：顶部的返回按钮点击后能够迅速返回到上一级页面。

选项按钮：页面中的每一个选项都是一个可点击的按钮。`optionItem` 组件提供了图标、文本和导航箭头的组合，提示用户该选项可以点击并跳转到下一页面。

3.3.11.5 关键函数

`Scroll()`:

函数负责页面内容的滚动行为，包裹在 `Scroll` 中的 `Column` 容器使页面内容能够在垂直方向上滚动。使用 `Scroller` 对象来管理滚动行为，提升了长页面的可读性和用户体验。

3.3.11.6 页面功能

代码的主要功能是展示一系列设置选项，通过 `optionItem` 组件渲染这些选项。每个选项都带有图标和文字，模拟 QQ 点击选项可以进入相应的设置页面的行为。

通过使用 `Scroll` 容器，页面能够很好地处理大量设置项的展示问题，用户可以滚动页面查看所有选项，并进行操作。

通过调整不同选项之间的间距（如使用不同的 `marginTop` 值），页面呈现出层次分明的视觉效果，避免了内容的密集堆积，使用户在浏览时更加轻松。



3.3.12 聊天页面实现

3.3.12.1 涉及的文件

src > main > ets > pages > Message.ets

src > main > ets > common > components > navigationBar.ets

src > main > ets > common > components > listitem > MessageListItem.ets

src > main > ets > common > components > messageBottom.ets

src > main > ets > model > data > MessageData.ets

src > main > ets > model > data > SessionData.ets

src > main > ets > model > MessageModel.ets

3.3.12.2 文件源代码

Message.ets 代码：

```

1. // 该组件为聊天页面

```

```

2. import { navigationBar } from '../common/components/navigationBar'
3. import { MessageData } from '../model/data/MessageData'
4. import { SessionData } from '../model/data/SessionData'
5. import { getMessagesDatas } from '../model/MessageModel'
6. import { MessageListItem } from '../common/components/listitem/MessageListItem'
7. import { messageBottom } from '../common/components/messageBottom'
8. import router from '@ohos.router';
9. interface RouterParams {
10.   sessionData: SessionData;
11. }
12. @Entry
13. @Component
14. struct Message {
15.   private msgsItems: MessageData[] = getMessagesDatas()
16.   private sessionData: SessionData = (router.getParams() as RouterParams).sessionData
17.   build() {
18.     Flex({ direction: FlexDirection.Column, alignItems: ItemAlign.Start, justifyContent: FlexAlign.SpaceBetween }) {
19.       navigationBar({title: this.sessionData.name, isBack:true , isClose:false , isMenu: false , isMain: false , isMore:true})
20.       List(){
21.         ForEach(this.msgsItems, (item: MessageData) => {
22.           ListItem() {
23.             MessageListItem({ msgItem: item })
24.           }
25.           }, (item: MessageData) => item.mid)
26.         }.padding({top:10 , bottom: 10})
27.         .width('100%')
28.         messageBottom()
29.       }
30.       .width('100%')
31.       .height('100%')
32.       .backgroundColor($r('app.color.grey0'))
33.     }
34.   }

```

MessageListItem.ets 代码:

```

1. // 聊天聊天记录 List 的 item 布局
2. import { msgIO , msgType , MessageData } from '../../model/data/MessageData'
3. import router from '@ohos.router';
4. interface ImageSize {
5.   width: number;
6.   height: number;
7. }

```

```

8. @Component
9. export struct MessageListItem{
10.   private msgItem: MessageData = new MessageData(
11.     'default_mid',
12.     'default_qqid',
13.     msgIO.Send,
14.     'default_context',
15.     msgType.Text,
16.     Date.now(),
17.     'default_avatar'
18.   )
19.   @State imgHeight: number = 0
20.   @State imgWidth: number = 0
21.   @Builder header_layout() {
22.     Flex({ direction: FlexDirection.Row , justifyContent: FlexAlign.Center , alignI
      tems:
23.       ItemAlign.Center}){
24.       Image($rawfile(this.msgItem.avatar))
25.       .borderRadius(100)
26.       .width(50).height(50)
27.     }.height(60).width(60).onClick(() => {
28.       router.pushUrl({
29.         url: 'pages/UserInfo',
30.         params: {
31.           contactData: {
32.             qqid: this.msgItem.qqid,
33.             headImg: this.msgItem.avatar,
34.             name: '非门控通道',
35.             mark: '非门控通道',
36.             area: '河北·保定'
37.           }
38.         }
39.       });
40.     });
41.   }
42. //发送消息布局
43. @Builder sendMsgLayout(){
44.   Flex({ direction: FlexDirection.RowReverse , justifyContent: FlexAlign.End}){
45.     //头像
46.     this.header_layout()
47.     //消息内容
48.     Flex({ direction: FlexDirection.Row , justifyContent: FlexAlign.End , alignIte
      ms: ItemAlign.Center}){
49.       if(this.msgItem.type == msgType.Text){ //文本消息
50.         Text(this.msgItem.context)
51.         .backgroundColor($r('app.color.light_blue3'))

```

```

52.         .borderRadius(4)
53.         .padding(8)
54.         .fontFamily($r('sys.float.ohos_id_text_size_body1'))
55.         .fontSize(22)
56.         .fontColor($r('app.color.white'))
57.         Image($r('app.media.right_arrow'))
58.         .height(12)
59.         .width(6)
60.     }else if(this.msgItem.type == msgType.Pic){    //图片消息
61.         //Flex(){
62.         Image($rawfile(this.msgItem.context))
63.         .objectFit(ImageFit.Contain)
64.         .borderRadius(4)
65.         .sourceSize({width:this.imgWidth , height: this.imgHeight })
66.         .onComplete((msg: ImageSize) => {
67.             this.imgWidth = msg.width
68.             this.imgHeight = msg.height
69.         })
70.         Image($r('app.media.right_arrow'))
71.         .height(12)
72.         .width(6)
73.     }else if(this.msgItem.type == msgType.Sound){ //语音消息
74.         Flex({direction: FlexDirection.RowReverse , alignItems: ItemAlign.Center}
75.     ){
76.         Image($r('app.media.ic_sound_left'))
77.         .height(20)
78.         .width(20)
79.         .margin({right: 5})
80.         Text('4\''')
81.         .fontFamily($r('sys.float.ohos_id_text_size_body1'))
82.         .fontSize(16)
83.         .margin({right: 5})
84.         }.height(46).width(80).padding(6).backgroundColor($r('app.color.light_blue3')).margin({top: 6})
85.         Image($r('app.media.right_arrow'))
86.         .height(12)
87.         .width(6)
88.     }
89.     }.layoutWeight(1)
90.     Blank().width(78)
91. }.width('100%').margin({bottom: 20})
92. //接收消息布局
93. @Builder recvMsgLayout(){
94.     Flex({ direction: FlexDirection.Row , justifyContent: FlexAlign.Start}){
95.         //头像

```

```

96.     this.header_layout()
97.     //消息内容
98.     Flex({ direction: FlexDirection.RowReverse , justifyContent:FlexAlign.End , alignItems:ItemAlign.Center}){
99.         if(this.msgItem.type == msgType.Text){           //文本消息
100.             Text(this.msgItem.context)
101.                 .backgroundColor($r('app.color.white'))
102.                 .borderRadius(4)
103.                 .padding(8)
104.                 .fontFamily($r('sys.float.ohos_id_text_size_body1'))
105.                 .fontSize(22)
106.             Image($r('app.media.left_arrow'))
107.                 .height(12)
108.                 .width(6)
109.         }else if(this.msgItem.type == msgType.Pic){       //图片消息
110.             //Flex(){
111.             Image($rawfile(this.msgItem.context))
112.                 .objectFit(ImageFit.Contain)
113.                 .borderRadius(4)
114.                 .sourceSize({width:this.imgWidth , height: this.imgHeight })
115.                 .onComplete((msg: ImageSize) => {
116.                     this.imgWidth = msg.width
117.                     this.imgHeight = msg.height
118.                 })
119.             Image($r('app.media.left_arrow'))
120.                 .height(12)
121.                 .width(6)
122.         }else if(this.msgItem.type == msgType.Sound){ //语音消息
123.             Flex({direction: FlexDirection.Row , alignItems: ItemAlign.Center}){
124.                 Image($r('app.media.ic_sound_right'))
125.                     .height(20)
126.                     .width(20)
127.                     .margin({left: 5})
128.                 Text('4\''')
129.                     .fontFamily($r('sys.float.ohos_id_text_size_body1'))
130.                     .fontSize(16)
131.                     .margin({left: 5})
132.             }.height(46).width(80).padding(6).backgroundColor($r('app.color.white'))
133.             .margin({top: 6})
134.             Image($r('app.media.left_arrow'))
135.                 .height(12)
136.                 .width(6)
137.         }
138.         .layoutWeight(1)
139.         Blank().width(78)
140.     }.width('100%').margin({bottom: 20})

```



```

140. }
141. build(){
142.     if(this.msgItem.io == msgIO.Send){
143.         this.sendMsgLayout()
144.     }else if(this.msgItem.io == msgIO.Recv){
145.         this.recvMsgLayout()
146.     }
147. }
148. }

```

messageBottom.ets 代码:

```

1. //聊天页底部输入栏布局
2. @Component
3. export struct messageBottom {
4.     build() {
5.         Column(){
6.             Row() {
7.                 TextInput()
8.                     .type(InputType.Normal)
9.                     .enterKeyType(EnterKeyType.Send)
10.                    .caretColor($r('app.color.green0'))
11.                    .borderRadius(5)
12.                    .fontSize(16)
13.                    .fontFamily($r('sys.float.ohos_id_text_size_body1'))
14.                    .backgroundColor($r('app.color.white'))
15.                    .width('70%')
16.                    .height(40)
17.                 Button($r('app.string.fs'), { type: ButtonType.Normal })
18.                    .backgroundColor($r("app.color.light_blue2"))
19.                    .fontSize(20)
20.                    .fontColor($r("app.color.white"))
21.                    .borderRadius(5)
22.                    .margin({left: 10})
23.             }
24.             Image($r('app.media.bt1'))
25.                 .width('100%')
26.         }
27.         .width('100%')
28.         .height(80)
29.     }
30. }

```

MessageData.ets 代码:

```

1. //聊天记录结构

```

```

2. export enum msgIO {
3.     //发送
4.     Send ,
5.     //接收
6.     Recv
7. }
8. export enum msgType{
9.     //文本消息
10.    Text ,
11.    //图片消息
12.    Pic ,
13.    //语音消息
14.    Sound
15. }
16. export class MessageData{
17.    mid: string        //消息唯一 id
18.    qqid: string       //用户 qqid
19.    io: msgIO          //消息流动方向
20.    context: string    //消息内容
21.    type: msgType      //消息类型
22.    time: number       //时间戳
23.    avatar: string     //头像
24.    constructor(mid:string , qqid: string , io: msgIO , context: string , type: msgTy
        pe, time: number, avatar: string){
25.        this.mid = mid
26.        this.qqid = qqid
27.        this.io = io
28.        this.context = context
29.        this.type = type
30.        this.time = time
31.        this.avatar = avatar
32.    }
33. }

```

MessageModel.ets 代码:

```

1. // 聊天页数据
2. import { msgIO , msgType , MessageData } from './data/MessageData'
3. export function getMessagesDatas(): Array<MessageData> {
4.    let msgsDataArray: Array<MessageData> = []
5.    MessagesComposition.forEach((item: MessageItem) => {
6.        msgsDataArray.push(new MessageData(item.mid , item.qqid , item.io , item.conte
            xt , item.type , item.time, item.avatar))
7.    })
8.    return msgsDataArray;
9. }

```

```

10. interface MessageItem {
11.     mid: string;
12.     qqid: string;
13.     io: msgIO;
14.     context: string;
15.     type: msgType;
16.     time: number;
17.     avatar: string;
18. }
19. const MessagesComposition: MessageItem[] = [
20.     {
21.         mid: "1",
22.         qqid: "1627018151",
23.         io: msgIO.Send,
24.         context: "轻轻的我走了, \n" +
25.             "正如我轻轻的来; \n" +
26.             "我轻轻的招手, \n" +
27.             "作别西天的云彩。",
28.         type: msgType.Text,
29.         time: 1,
30.         avatar: "qq0.jpg"
31.     },
32.     {
33.         mid: "2",
34.         qqid: "2830133296",
35.         io: msgIO.Recv,
36.         context: "那河畔的金柳, \n" +
37.             "是夕阳中的新娘; \n" +
38.             "波光里的艳影, \n" +
39.             "在我的心头荡漾。",
40.         type: msgType.Text,
41.         time: 2,
42.         avatar: "qq1.jpg"
43.     },
44.     {
45.         mid: "3",
46.         qqid: "1627018151",
47.         io: msgIO.Send,
48.         context: "p1.jpg",
49.         type: msgType.Pic,
50.         time: 3,
51.         avatar: "qq0.jpg"
52.     },
53.     {
54.         mid: "4",
55.         qqid: "2830133296",

```

```

56.   io: msgIO.Recv,
57.   context: "p2.jpg",
58.   type: msgType.Pic,
59.   time: 4,
60.   avatar: "qq1.jpg"
61. },
62. {
63.   mid: "5",
64.   qqid: "1627018151",
65.   io: msgIO.Send,
66.   context: "sample_sound.mp3",
67.   type: msgType.Sound,
68.   time: 5,
69.   avatar: "qq0.jpg"
70. },
71. {
72.   mid: "6",
73.   qqid: "2830133296",
74.   io: msgIO.Recv,
75.   context: "sample_sound.mp3",
76.   type: msgType.Sound,
77.   time: 6,
78.   avatar: "qq1.jpg"
79. }
80. ]

```

3.3.12.3 布局结构

聊天页面主要分为三个部分：顶部的导航栏、中间的消息展示区域和底部的输入栏。

导航栏使用 `navigationBar` 组件设置了页面标题，并包含返回按钮（`isBack: true`）和更多操作按钮（`isMore: true`）。这个导航栏固定在页面顶部，方便用户快速返回或查看更多功能。

页面的主体部分是消息列表，通过 `List` 组件展示聊天记录。每条消息项由 `MessageListItem` 组件渲染，支持文本、图片、语音等不同类型的消息展示，并自动区分消息的发送和接收方向。

底部部分是一个固定的输入区域，使用 `messageBottom` 组件实现，包含一个输入框和发送按钮，用户可以在此输入消息并发送。

3.3.12.4 按钮设计

导航按钮：用户可以通过导航栏中的返回按钮和更多按钮返回上一页面或者查看更多选项。

消息项按钮：在消息列表中，每个消息项实际上是一个按钮。用户点击头像可以进入用户详情页面，提升了应用的易用性。

输入栏按钮：输入栏包含一个发送按钮，模拟用户在输入消息后发送消息的行为。

3.3.12.5 关键函数

`getMessagesDatas()`:

这是一个数据获取函数，位于 `MessageModel.ets` 文件中，负责从静态数据源中获取



聊天记录数据，并将这些数据展示在聊天界面上。该函数通过返回一个包含 `MessageData` 对象的数组，为消息列表提供了动态的数据来源。

`messageBottom()`:

这是底部输入栏的组件函数，负责生成输入框和发送按钮的布局。这个函数通过整合输入与发送的功能，确保消息的输入与发送操作能够在一个固定的区域内完成。

3.3.12.6 页面功能

页面的核心功能是展示用户之间的聊天记录。通过 `MessageListItem` 组件，页面能够渲染不同类型的消息（如文本、图片、语音），并根据消息的发送方向，自动调整消息的布局。聊天记录的展示还包括了时间戳和用户头像，确保聊天内容的上下文清晰明确。

底部的 `messageBottom` 组件模拟用户输入新消息并发送，这部分功能与消息列表紧密配合，模拟了完整的聊天流程。

聊天记录数据通过 `MessageModel` 文件中的静态样例数据进行加载，并且 `MessageData` 类为每条消息的存储提供了统一的结构。这种设计确保了灵活性，未来可以轻松扩展和修改数据内容。

3.3.13 好友资料页面实现

3.3.13.1 涉及的文件

`src > main > ets > pages > UserInfo.ets`

`src > main > ets > common > components > navigationBar.ets`

`src > main > ets > model > data > ContactData.ets`

3.3.13.2 文件源代码

```
1. //用户信息页面
2. import router from '@ohos.router';
3. import { navigationBar } from '../common/components/navigationBar'
4. import { ContactData , Gender } from '../model/data/ContactData'
5. interface RouterParams {
6.     contactData: ContactData;
7. }
8. @Entry
9. @Component
10. struct UserInfo {
11.     private contactData: ContactData = (router.getParams() as RouterParams).contactData;
12.     build() {
13.         Column(){
14.             navigationBar({title: '个人资料', isBack: true , isClose: false , isMenu: false , isMain: false , isMore: true , backgroundColor: $r('app.color.white')})
15.             Row(){
16.                 Image($rawfile(this.contactData.headImg))
17.                 .width(100).height(100)
18.                 .borderRadius(100)
19.                 Flex({direction: FlexDirection.Column , justifyContent: FlexAlign.Center , alignItems: ItemAlign.Start}){
```

```

20.         Row(){
21.             Text(this.contactData.name)
22.                 .fontFamily($r('sys.float.ohos_id_text_size_sub_title1'))
23.                 .fontColor($r('app.color.black'))
24.                 .fontSize(22)
25.                 .maxLines(1)
26.                 .fontWeight(FontWeight.Bold)
27.         }
28.         Row(){
29.             Text($r('app.string.info_qqid'))
30.                 .fontFamily($r('sys.float.ohos_id_text_size_body1'))
31.                 .fontColor($r('app.color.grey3'))
32.                 .fontSize(14)
33.                 .maxLines(1)
34.             Text(this.contactData.qqid)
35.                 .fontFamily($r('sys.float.ohos_id_text_size_body1'))
36.                 .fontColor($r('app.color.grey3'))
37.                 .fontSize(14)
38.                 .maxLines(1)
39.         }.margin({top: 6})
40.         Row(){
41.             Text($r('app.string.info_area'))
42.                 .fontFamily($r('sys.float.ohos_id_text_size_body1'))
43.                 .fontColor($r('app.color.grey3'))
44.                 .fontSize(14)
45.                 .maxLines(1)
46.             Text(this.contactData.area)
47.                 .fontFamily($r('sys.float.ohos_id_text_size_body1'))
48.                 .fontColor($r('app.color.grey3'))
49.                 .fontSize(14)
50.                 .maxLines(1)
51.         }.margin({top: 4})
52.     }.height(160).layoutWeight(1)
53.     .padding(20)
54.     Image($r('app.media.dz0'))
55.         .width(30).height(30)
56.     }
57.     .width('100%').height(160)
58.     .padding({left: 20 , right: 20})
59.     .alignItems(VerticalAlign.Center)
60.     .backgroundColor($r('app.color.white'))
61.     Divider().color($r('app.color.grey2')).strokeWidth(0.8).margin({left:10 , right: 10})
62.     Column(){
63.         Image($r('app.media.info'))
64.         .width('100%')

```

```

65.     }
66.     Flex({justifyContent:FlexAlign.SpaceBetween}){
67.         Button($r('app.string.inf1') , { type: ButtonType.Normal })
68.             .width(150)
69.             .height(50)
70.             .backgroundColor($r("app.color.white"))
71.             .fontSize(16)
72.             .fontColor($r("app.color.black"))
73.             .borderRadius(10)
74.             .padding({ top: '0.00vp', right: '16.00vp', bottom: '0.00vp', left: '16.0
0vp' })
75.             .borderWidth('2.00vp')
76.             .borderColor('#7a000000')
77.         Button($r('app.string.inf2') , { type: ButtonType.Normal })
78.             .width(150)
79.             .height(50)
80.             .backgroundColor($r("app.color.white"))
81.             .fontSize(16)
82.             .fontColor($r("app.color.black"))
83.             .borderRadius(10)
84.             .padding({ top: '0.00vp', right: '16.00vp', bottom: '0.00vp', left: '16.0
0vp' })
85.             .borderWidth('2.00vp')
86.             .borderColor('#7a000000')
87.         Button($r('app.string.inf3'), { type: ButtonType.Normal })
88.             .width(150)
89.             .height(50)
90.             .fontSize(16)
91.             .fontColor($r("app.color.white"))
92.             .borderRadius(10)
93.             .backgroundImageSize({ width: '', height: '' })
94.             .backgroundColor('#ff009aff')
95.             .onClick(() => {
96.                 router.pushUrl({
97.                     url: 'pages/Message',
98.                     params: { sessionData: this.contactData } // 传递参数
99.                 });
100.            });
101.     }
102. }
103. .width('100%')
104. .height('100%')
105. .backgroundColor($r('app.color.grey2'))
106. }

```

3.3.13.3 布局结构

顶部导航栏:使用 `navigationBar` 组件,设置了返回按钮(`isBack: true`),并且为导航栏指定了白色背景 (`bgColor: $r('app.color.white')`),导航栏固定在页面顶部。

用户信息展示区:通过 `Row` 容器展示用户的头像、姓名、QQ 号和地区等信息。头像位于左侧,用户的详细信息通过多个 `Row` 容器纵向排列。页面右上角还放置了一个图标按钮,整个信息展示区采用白色背景。

操作按钮区:页面底部使用 `Flex` 容器,均匀排列了三个操作按钮,用户可以选择与该联系人进行的具体操作,如发送消息。

3.3.13.4 按钮设计

页面底部有三个操作按钮,这些按钮的设计保持了一致的风格,包含统一的宽度、高度、圆角和颜色配置。按钮的点击区域宽大,保证了用户操作时的便利性。其中一个按钮(发送消息按钮)附带点击事件,点击后会跳转到聊天页面。

每个按钮在点击时都会有明确的视觉反馈,确保用户能及时了解操作结果,提升操作体验。

3.3.13.5 关键函数

`router.pushUrl()`:该函数负责页面的导航功能,例如当用户点击发送消息按钮时,`router.pushUrl()`函数被调用,将用户引导至聊天页面,并将当前用户的资料作为参数传递过去。这个函数是页面之间流畅导航的关键。

3.3.13.6 页面功能

页面核心功能是展示用户的详细资料,包括头像、昵称、QQ 号和地区信息。这些信息通过整齐的排版和适当的间距,确保了用户可以快速获取所需的个人信息。

通过导航按钮和页面跳转逻辑,用户可以在查看资料后返回上一级页面,或者跳转到聊天页面与该联系人进行交流。页面底部的操作按钮提供了与该用户进一步互动的入口。

通过传递 `contactData` 参数,页面可以在不同模块之间共享用户数据,实现了用户资料与其他功能的无缝对接。



3.3.14 空间动态页面实现

3.3.14.1 涉及的文件

`src > main > ets > pages > Moments.ets`

3.3.14.2 文件源代码

```
1. // 该组件为空间动态页面
2. import router from '@ohos.router';
3. @Entry
4. @Component
5. struct Moments {
6.   build(){
7.     Column(){
8.       Stack({ alignContent: Alignment.Top }){
```



```

9.         Image($rawfile('gradualchange.png'))
10.         .objectFit(ImageFit.Cover)
11.         .height(250)
12.         .width('100%')
13.         Flex({direction: FlexDirection.Row , justifyContent: FlexAlign.SpaceBetween
, alignItems: ItemAlign.Center}){
14.             Image($r('app.media.ic_back_white'))
15.             .height(30)
16.             .width(30)
17.             .margin({ left: 15})
18.             .onClick(() => {
19.                 router.back()
20.             })
21.             Text('好友动态')
22.             .fontColor($r('app.color.white'))
23.             .fontSize(20)
24.             .fontWeight(FontWeight.Bold)
25.             Image($r('app.media.ic_add'))
26.             .height(30)
27.             .width(30)
28.             .margin({ right: 15})
29.         }.width('100%').height(60).zIndex(1)
30.         Image($rawfile('qq0.jpg'))
31.         .width(75).height(75)
32.         .borderRadius(100)
33.         .margin({top: 140,right: 200})
34.         Flex({direction: FlexDirection.Row , justifyContent: FlexAlign.End , alignI
tems: ItemAlign.End}){
35.             Text('今日访客 0')
36.             .margin({bottom: 80 , right: 30})
37.             .fontFamily($r('sys.float.ohos_id_text_size_body1'))
38.             .fontColor($r('app.color.white'))
39.             .fontSize(18)
40.             .maxLines(1)
41.         }.width('100%').height(280)
42.     }.width('100%').height(280)
43.     Column(){
44.         Image($rawfile('mo0.jpg'))
45.         .width('100%')
46.         Image($rawfile('mo1.jpg'))
47.         .width('100%')
48.     }
49. }
50. }
51. }

```

3.3.14.3 布局结构

空间动态页面的设计主要分为顶部区域和内容展示区域。

页面顶部使用了 Stack 容器，背景是全宽图片，设置了图片覆盖（ImageFit.Cover）效果，使得整个区域充满视觉冲击力。

在顶部背景图片之上，使用了 Flex 容器进行布局，包含了三个主要元素：返回按钮、页面标题和右侧的添加按钮。返回按钮和添加按钮对称布局，标题文本位于中央。

用户头像图片放置在背景图片的中下部，通过 margin 调整其位置。

页面底部展示"今日访客"，通过 Flex 容器定位在页面底部的右侧。

3.3.14.4 页面功能

页面的主要功能是展示好友的动态，分为顶部区域和内容区域，两个部分各自承担不同的功能。顶部区域侧重于用户信息和导航，内容区域则是动态的核心展示区。

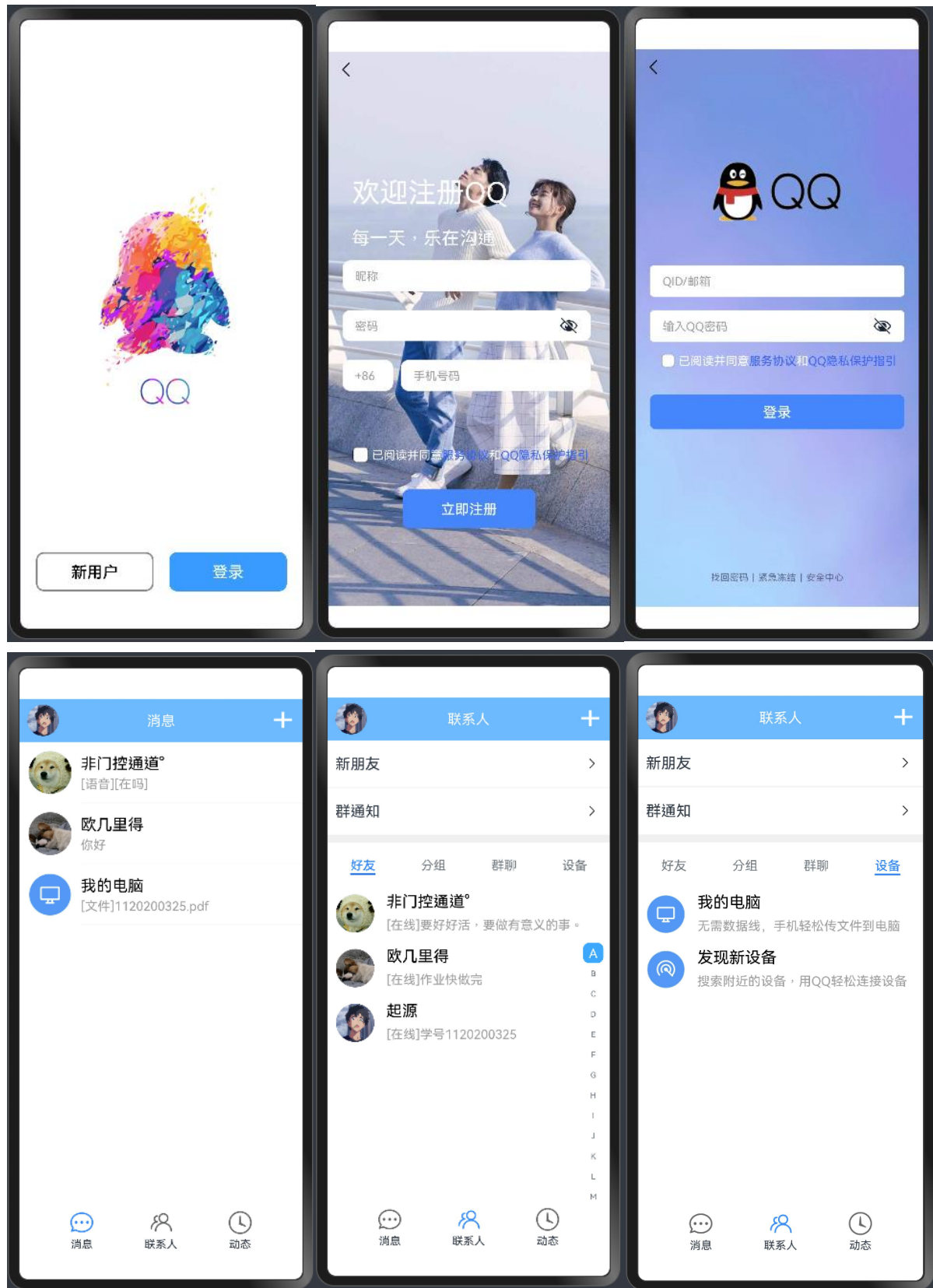


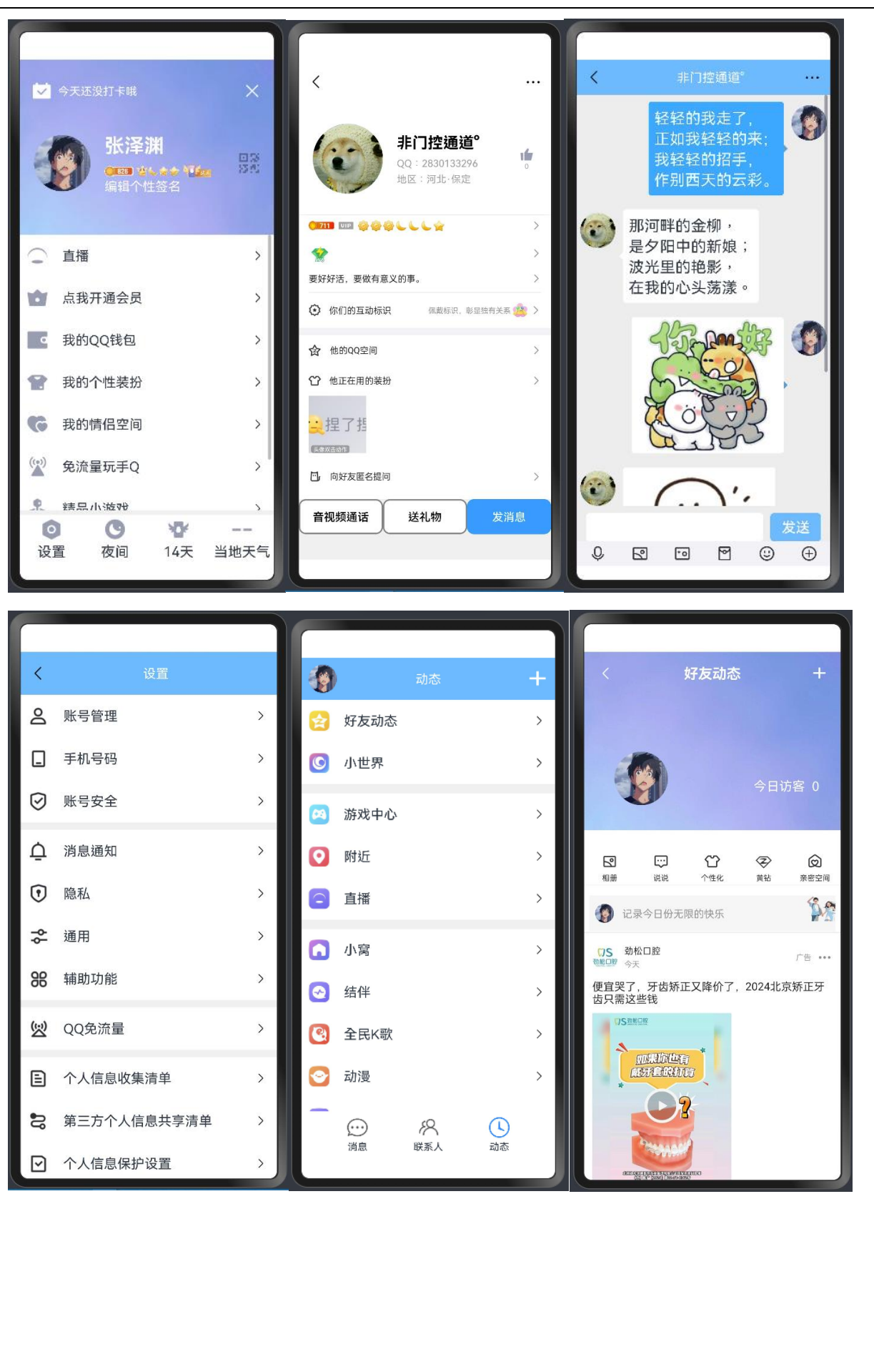
3.3.15 备注

实验代码中涉及的用户资料均取材于腾讯 QQ 用户的真实资料，包括用户的 QQ 号、昵称、头像等，使用这些资料前已经征得了相关用户的同意。

四、实验结果

将代码进行编译后可以得到以下页面：





各个页面之间的跳转关系如下图所示：



软件的演示视频可通过访问此链接观看：

<https://www.bilibili.com/video/BV1eJWKeoEqm/>

五、实验感想

通过本次暑期特训，开发基于 OpenHarmony 的 QQ 聊天软件模仿应用，我从多个方面收获颇丰。暑期特训的整个过程无论是在学习的广度还是深度上，都给予了我全面的锻炼与挑战，促使我不断反思并改进自己的开发方式。

首先，在技术层面上，本次实习为我提供了难得的实践机会，使我有机会深入了解并熟练掌握了 DevEco Studio 开发环境以及 ArkTS 语言的应用。OpenHarmony 作为一款新兴的操作系统，其开发架构与传统的 Android 或 iOS 存在一定的差异，这给我带来了全新的学习曲线。在环境搭建初期，我遇到了一些困难，特别是在开发环境的配置和调试环节，我花费了较多的时间去研究相关文档来解决问题。这一过程不仅让我掌握了如何在 OpenHarmony 上进行开发，也让我学会了在面对技术问题时保持耐心和细致，逐步排查和解决问题。

在软件架构设计方面，我从实际需求出发，深入思考了应用的整体结构如何规划才能既保证代码的简洁性，又能够扩展更多的功能。在这次开发中，组件化编程的理念对我影响深远。我意识到，应用程序的开发不仅仅是实现一个功能，还需要从长远考虑其可维护性和可扩展性。例如，我在开发多个页面时，设计了通用的组件模板，这样既避免了重复劳动，也提高了代码的复用性和模块化管理的效率。具体来说，导航栏、按钮、消息列表等模块都被我封装成了可复用的组件，使得这些功能能够在不同页面之间灵活调用，从而大大提升了开发效率和代码的一致性。

在编码实现过程中，我对界面设计的细节投入了大量时间和精力。作为一款模仿 QQ 界面的聊天软件，需要仔细控制各个组件的外观和位置。因此，我不仅关注了页面的整体布局，还仔细设计了每个交互细节。通过学习各个容器布局技术，我能够灵活控制页面元素的排列方式，确保在不同设备和屏幕尺寸下应用能够保持良好的响应效果。同时，在按钮、列表项等交互元素的设计中，我采用了不同的点击反馈效果，使得用户在进行操作

时能够获得直观的视觉反馈。这些设计不仅提升了用户体验，也让我对前端开发中的交互设计有了更深入的理解。 总而言之，这次实习为我提供了一个非常宝贵的实践机会，不仅在技术上让我获得了突破，还让我对软件开发的整个流程有了更加全面和深刻的理解。通过这次实习，我不仅提高了编程技能，更加明确了未来的职业方向，也更加坚定了我软件工程道路上不断前行的信心与决心。	
指导老师 审核意见	指导老师签字： 年 月 日
学院 审核意见	主管签字（盖章）： 年 月 日